技术文件

完成时间：2024.12.29

工程实践与科技创新[3A]
设计报告

2024 年

项目名称： DC-DC 开关电源系统设计

设计小组编号：24

设计小组名单：余秉航(组长)、张伯言、林子曦

上海交通大学 电子信息与电气工程学院

姓名	班级	学号	具体负责的工作	联系方式
余秉航	2244	522031910179	硬件焊装、报告撰写	13996513307
张伯言	2244	522031910205	PCB 设计、报告撰写	15213559066
林子曦	2242	522031910593	软件设计、报告撰写	13922363780



上海交通大学 电子信息与电气工程学院

地　址：东川路 800 号

邮　编：200240

摘　要：

本报告描述上海交通大学 2022 级工程实践与科技创新IIIA 的实验项目全过程。该实验项目基于 TPS40200 芯片设计并实现 DC-DC 开关稳压供电及稳流供电系统，同时借助 TM4C1294NCPDT 单片机对输出电流与输出电压进行测量和显示，并最终实现并联均流输出。稳压供电源输出电压约为 5V，并带有输出过流保护；稳流供电源输出电流设定为 0~1V，可通过单片机结合 DAC 芯片进行控制，并带有输出过压保护功能。

本报告包含如下部分：系统总述、硬件电路原理设计、参数设计、单片机编程与调试。报告附录写有系统操作说明书、测试和分析、课程心得以及程序代码清单等内容。

关键词：

TPS40200 芯片，DC-DC 开关电源，TM4C1294NCPDT 单片机，并联均流输出

ABSTRACT

This report describes the entire process of the experimental project of Engineering Practice and Technological Innovation IIIA for the 2022 grade at Shanghai Jiao Tong University. The experimental project is based on the TPS40200 chip to design and implement a DC-DC switching regulated power supply and constant current power supply system. Meanwhile, the TM4C1294NCPDT microcontroller is used to measure and display the output current and voltage, and ultimately achieve parallel current sharing output. The regulated power supply output voltage is approximately 5V and has output overcurrent protection. The constant current power supply output current is set to 0-1V and can be controlled by the microcontroller in combination with the DAC chip, and it also has output overvoltage protection function.

This report includes the following parts: system overview, hardware circuit principle design, parameter design, microcontroller programming and debugging. The appendix of the report contains the system operation manual, test and analysis, course reflection, and program code list, etc.

KEYWORDS

TPS40200 chip，DC-DC switching power supply，TM4C1294NCPDT，parallel current sharing output

---

上海交通大学 电子信息与电气工程学院

地　址： 东川路 800 号

邮　编： 200240

目　　录

1. 概述 1

1.1 编写说明 1

1.2 名词定义 1

1.3 硬件开发环境 2

1.4 软件开发环境 2

1.5 缩略语 2

2. 系统总述 3

2.1 系统组成 3

2.2 系统的主要功能 3

2.2.1 DC-DC 开关稳压电源模块 3

2.2.2 DC-DC 开关稳流电源模块 3

2.2.3 单片机程控模块 3

3. DC-DC 开关稳压供电系统的设计和实现 4

3.1 主要功能和设计指标 4

3.1.1 主要功能 4

3.1.2 设计指标 4

3.2 电路设计原理 5

3.2.1 TPS40200 的工作原理 5

3.2.2 电路模块设计 5

3.3 电路重要部分设计与参数确定 6

3.3.1 开关工作频率 6

3.3.2 设置软启动过程时间 6

3.3.3 输出电压 6

3.3.4 输出限流保护设点 7

3.3.5 电流检测信号调理电路 7

3.3.6 电压检测信号调理电路 7

3.4 输出电压与输出电流的单片机测量 7

3.5 关键问题与解决 8

3.5.1 输出纹波 8
3.5.2 电源转换效率 9
4. DC-DC 开关稳流供电系统的设计和实现 10
4.1 主要功能和设计指标 10
4.1.1 主要功能 10
4.1.2 设计指标 10
4.2 电路设计原理 10
4.3 电路重要部分设计与参数确定 12
4.3.1 DAC6571 控制模块 12
4.3.2 误差放大电路 13
4.3.3 限压保护电路 13
4.3.4 其他电路修改 14
4.4 标定和修正 14
4.4.1 输出电流与其 ADC 转换编码之间的关系（关系式 A） 14
4.4.2 输出电压与其 ADC 转换编码之间的关系（关系式 B） 17
4.4.3 DAC 编码与稳流设点之间的关系（关系式 C） 17
4.5 单片机闭环控制式稳流算法 18
4.5.1 状态 1：稳流粗设 18
4.5.2 状态 2：延时 19
4.5.3 状态 3：稳流动态细调 20
4.5.4 状态 4：过压保护 21
4.6 关键问题解决 21
4.6.1 电源效率提高 21
5. 开关稳流电源模块的 PCB 设计 22
5.1 稳流源电路原理图纸的绘制 22
5.1.1 工作步骤 22
5.1.2 工作要领 22
5.2 稳流源电路 PCB 图纸的绘制 22
5.2.1 工作步骤 22

5.2.2 工作要领 23
6. 并联供电系统及模块负载均衡的设计和实现 24
6.1 主要功能和设计指标 24
6.1.1 主要功能 24
6.1.2 设计指标 24
6.2 电路设计原理 24
6.3 电路参数标定与修正 25
6.4 单片机程控算法 27
7. 致谢 29
8. 参考文献 30
9. 附录 A　系统操作说明书 31
9.1 系统功能 31
9.2 使用方法 31
9.3 稳压电源模式 31
9.4 稳流电源模式 31
9.5 并联式稳压供电模式 31
10. 附录 B　测试和分析 32
10.1 测试项目和方法 32
10.2 测试的资源 35
10.3 测试结果及分析 35
11. 附录 C　课程学习心得和意见建议 39
12. 附录 D　软件程序清单 40

1. 概述

1.1 编写说明

本报告为上海交通大学 2022 级工程实践与科技创新IIIA 的项目实验报告。该报告介绍了基于 TPS40200 设计并实现 DC-DC 开关稳压供电及稳流供电系统的方法，以及借助 TM4C1294NCPDT 单片机对输出电流与输出电压进行测量和显示，并最终实现并联均流输出的过程。

本报告包括系统总述、硬件电路原理设计、参数设计、单片机编程与调试等部分，报告附录写有系统操作说明书、课程心得以及程序代码清单等内容。报告旨在详细记录实验项目的设计思路与实践过程，分享实验的经验与心得，并以此为电子系学生群体提供相关参考。

1.2 名词定义

降压型 BUCK 结构 DC-DC 开关电源：采用 BUCK 电路结构，通过对开关器件的频繁切换来实现输入电压的降压过程。

负载调整率： 在负载电流发生变化时，开关稳压源输出电压的变化情况。

闭环控制： 闭环控制是一种通过反馈机制调整系统输出的控制方法。在闭环控制系统中，输出信号被实时监测并反馈到输入端，与预期输出进行比较，进而调整系统参数，使得实际输出接近设定值。

均流控制： 均流控制是指在并联多个电源时，通过调整各个电源的输出电流，使得所有电源的输出电流尽可能相等。通过均流控制，可以避免某一个电源过载，保证系统稳定运行。通常，均流控制需要通过反馈机制来调节电源输出的电流，使得电源间的负载分配更加均衡。

负载均衡度： 负载均衡度是衡量供电系统中各电源之间电流分配均匀程度的指标。较高的负载均衡度意味着电源系统能够更均匀地分配负载，减少过载和不均匀的电流负担，从而提升系统的整体性能和可靠性。

TPS40200： TPS40200 是一款宽输入非同步降压 DC-DC 控制电路。

DAC6571： DAC6571 是一款低功耗的、单通道的数模转换器。

TLV2372： TLV2372 是一款低功耗、双通道的运算放大器。

TL431： TL431 是一款可调精密并联稳压器，常用于产生稳定的 2.5V 参考电压。

1.3 硬件开发环境

硬件开发环境如表 1.1 所示：

开发工具	功能	运行环境
台式稳压源	提供 10V、20V 稳定直流电压	220V 交流电
示波器	展示并测量纹波与均衡响应时间	220V 交流电
数字万用表	测量输出电压、输出电流、电阻等指标	220V 交流电
电烙铁、焊锡、热风枪、吸焊器、镊子	焊接电路元件	电子实验室
Altim Designer	绘制电路图纸和 PCB 图纸	Windows 11

表 1.1 硬件开发环境

1.4 软件开发环境

软件开发环境如表 1.2 所示：

开发工具	功能	运行环境
Keil5 μVision5	单片机程序开发	Windows 11

1.5 缩略语

DC： direct current 直流电流

PWM： Pulse-Width Modulation 脉宽宽度调制

ADC： analog digital convert 模拟-数字转换

DAC： digital analog convert 数字-模拟转换

BUCK：Buck Converter 降压电路

2. 系统总述

2.1 系统组成

并联均流供电系统的组成框图如图 2.1 所示：

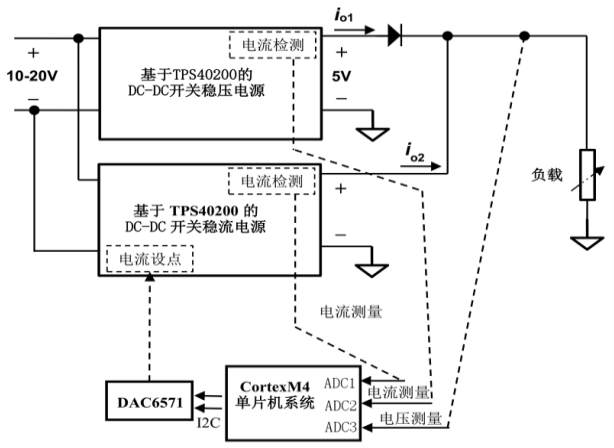


图 2.1 并联均流供电系统组成框图[1]

该系统由 DC-DC 开关稳压电源模块、DC-DC 开关稳流电源模块以及单片机程控模块组成。

2.2 系统的主要功能

系统功能的完善过程分为三个阶段：第一阶段完成 DC-DC 开关稳压电源模块，可输出约为 5V 直流稳定电压，并通过单片机测量显示输出电压电流；第二阶段将稳压电源模块改装为 DC-DC 开关稳流电源模块，使用单片机和 DAC 芯片组成的程控模块设置 0~1A 的输出电流；第三阶段两模块并联，稳压源输出的同时，其信号通过单片机程控模块设置稳流源输出，达到两模块并联均流输出。

2.2.1 DC-DC 开关稳压电源模块

该模块以 TPS40200 芯片为核心，可接受 10~20V 直流电压输入，并在不同负载条件下输出约 5V 的稳定直流电压，具有约 87%的电源转换效率和较低的输出纹波；同时，该模块电路设置有过流保护功能，在电流过大时使输出电压电流降至零，并可在电流减小后恢复正常工作。

单片机监测输出电流经差分放大电路、输出电压经分压电路调理后的信号，并在数码管显示。

2.2.2 DC-DC 开关稳流电源模块

该模块由 DC-DC 开关稳压电源模块改装完成，可接受 10~20V 直流电压输入，在不同负载条件下，由单片机控制输出 0.1~1A 电流，具有约 81%的电源转换效率；同时，该模块设置有过压保护功能，当输出电压超出阈值时，锁定输出电压和电流不再调整，待负载变化后恢复正常。

单片机可程控设置输出电流，并监测输出电压和电流信号，并在数码管显示。

2.2.3 单片机程控模块

该模块由 TM4C1294NCPDT 单片机和 DAC6571 组成，可通过 ADC 监测输出电压电流并显示在数码管；同时，可通过 DAC6571 程控设置输出电流。在两电源模块并联时，单片机程控模块监测稳压源输出信号，并通过 DAC6571 芯片控制稳流源输出，最终使两模块输出近似相等，实现并联均流输出功能。

3. DC-DC 开关稳压供电系统的设计和实现

3.1 主要功能和设计指标

3.1.1 主要功能

本模块为基于 TPS40200 芯片设计的 DC-DC 开关式稳压电源模块电路。该模块适配直流输入电压 10-20V，在该范围下稳定输出 5V 的直流电压，并带输出限流保护功能。该模块带有输出电流、电压的采样与信号调理电路，将信号调理或放大后输出至单片机进行检测。

该模块实物如图 3.1 所示。

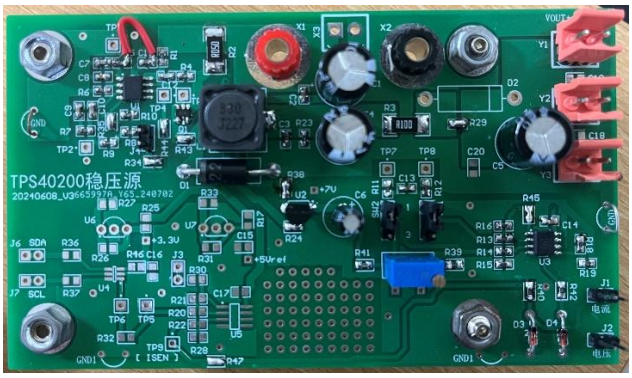


图 3.1 DC-DC 开关式稳压电源模块电路

3.1.2 设计指标

模块电路设计的技术指标如表 3.1 所示。

技术指标项	具体要求
输入电压	可适配直流 10-20V
额定输出电压	直流 5V
电压绝对精度	5V±3%
电压调整率	$\frac{\lvert 输入\ 20V\ 时的输出电压 - 输入\ 10V\ 时的输出电压 \rvert}{额定输出电压} \leq 0.5\%$（合格）
负载调整率	$\frac{\lvert 满载时的输出电压 - 半载时的输出电压 \rvert}{额定输出电压} \leq 0.5\%$（合格）
输出电压纹波	不大于 100mVpp（合格）
额定输出电流	1A
输出过流保护触发值允许范围	1.1~1.9A
电源变换效率（满载）	80%（合格）
空载时输出端应能正常测得电压	

表 3.1　电路设计技术指标及要求

3.2 电路设计原理

3.2.1 TPS40200 的工作原理

TPS40200 为宽输入非同步降压 DC/DC 控制器芯片。其功能原理如图 3.2 所示。

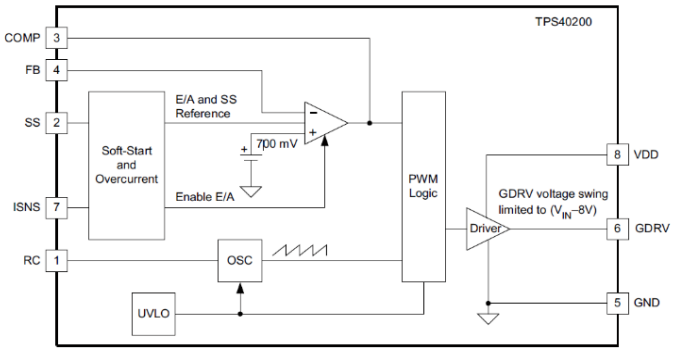


图 3.2 TPS40200 芯片功能原理图[2]

TPS40200 芯片的各引脚功能为[3]：

输入电压和欠压保护 UVLO（8 号脚）：欠压保护机制可以确保仅当输入电源电压已经超过正常工作所需最小电压时，芯片才正确启动。UVLO 门限为 4.25V。

软启动（2 号脚）：软启动作用于变换器控制电路，通常是通过逐渐增加脉冲占空比来实现的。软启动控制电路可使变换器从零启动，然后缓慢增加输出电压，减小了输出电容和变换器部件上的浪涌电流应力。

输出电压（3/4 号脚）：输出电压的设定是通过设计分压电路，将电源变换为输出端电压经分压，反馈给 TPS40200 芯片的误差放大器，再与内部基准电压 700mV 比较决定。

芯片的过流检测（7/8 号脚）：若 7、8 号引脚的压差超过 100mV，则开关晶体管断开，以此保护电路和负载。

工作频率（1/8 号脚）：外接 RC 电路与芯片内 OSC 模块生成锯齿波信号，可以定制工作频率

3.2.2 电路模块设计

DC-DC 开关式稳压电源模块电路大致可分为以下模块：

DC-DC 主体电路。为该模块的核心部分。

电压电流检测信号调理电路。用于调理或放大信号，供单片机测量。

辅助电源电路。为信号调理电路供电。

整体电路设计如图 3.3 所示。

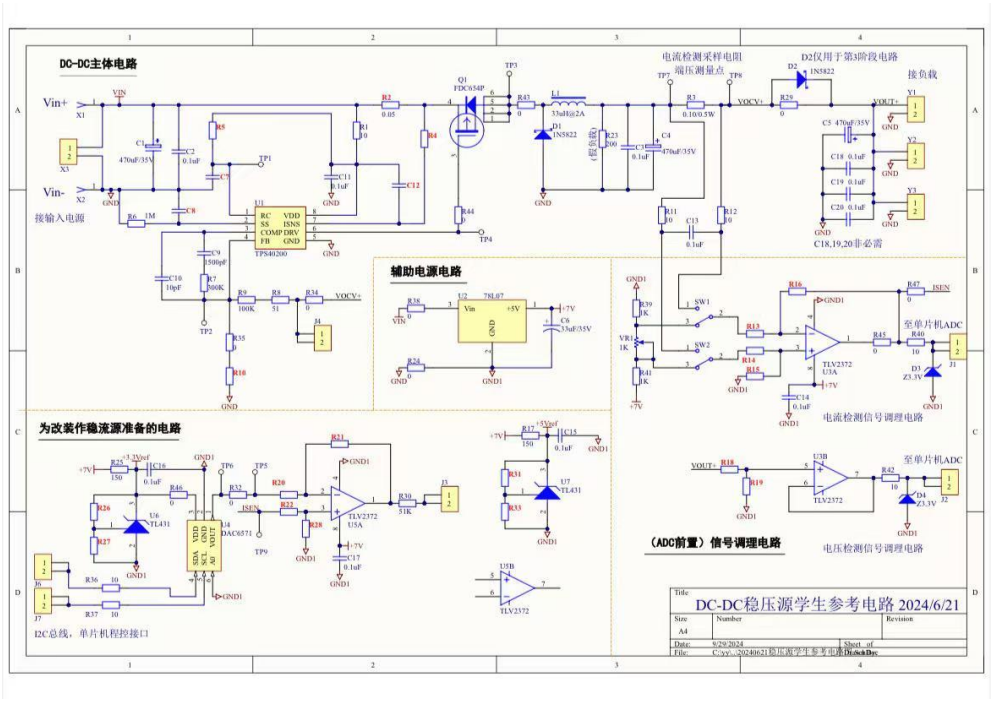


图 3.3 DC-DC 开关式稳压电源模块电路图[4]

3.3 电路重要部分设计与参数确定

3.3.1 开关工作频率

外围由 R5、C7 构成的 RC 电路，与芯片的 OSC 模块连接，以产生定制频率的周期性锯齿波，作为芯片的工作频率。振荡器频率计算公式如式(3.1)；为限制通过电容的电流，R5 还需满足式(3.2)。

$$f_{sw} = \frac{1}{R5 \times C7 \times 0.105} \qquad (3.1)$$

$$\frac{V_{in}}{R5} \leq 750\mu A \qquad (3.2)$$

取工作频率为 300kHz，故选取 R5 =68kΩ 以及 C7=470pF，此时振荡频率约为 298kHz。

3.3.2 设置软启动过程时间

为提高工程裕量，软启动时间不低于 30ms[5]。该电路中软启动时间计算公式如式(3.3)。

$$T_s = R_c \times C_8 \times \ln\left(\frac{V_{sst}}{V_{sst} - 1.4}\right) \qquad (3.3)$$

可知：$R_c = 105k\Omega$，$V_{sst} = 8V$。故选取 C8=2.2 μ F。

3.3.3 输出电压

输出电压涉及到电压反馈，主要由R_8、R_9、R_{10}、R_{34}、R_{35}组成的分压网络决定。可知 TPS40200 的四号脚在稳态时为 696mV[6]，且输出 VOCV+设定为 5V，故输出电压如式(3.4)

$$V_{OUT} = 0.696 \times \left(1 + \frac{R_8 + R_9 + R_{34}}{R_{10} + R_{35}}\right) \qquad (3.4)$$

故选取R_8=51Ω，R_9=100kΩ，R_{10}=16kΩ，R_{34}=R_{35}=0Ω。（关于零电阻：在实际焊接中，采用“堆锡”的方法将两个焊盘连接即可。）

3.3.4 输出限流保护设点

限流保护设点为 1.1A~1.9A。故取 R_2=0.068Ω，R_4=2kΩ，C_{12}=470μF。

本组的 R_2 取值为 0.05Ω，但效果亦较好。故该电阻最好通过后续过流保护的测试来选择。

3.3.5 电流检测信号调理电路

输出电流通过R_{13}、R_{14}、R_{15}和R_{16}与 TLV2372U3A 构成的差分放大电路，进行放大后再输入至单片机进行检测。出于对单片机的工作特性考量，差分放大倍数最好为 10~20 倍[7]。

放大倍数k满足式(3.5)。

$$k = \frac{R_{16}}{R_{13}} = \frac{R_{15}}{R_{14}} \qquad (3.5)$$

故选取R_{13}=R_{14}=10kΩ，R_{15}=R_{16}=150kΩ。

为减小误差，选择元件时，需要对多个元件的阻值进行测量，并选出差值最小的一对电阻。差值应小于千分之一。

3.3.6 电压检测信号调理电路

该信号调理电路通过 R_{18}、R_{19} 将输出电压分压，满足单片机模数转换的可接受范围；再通过 TLV2372 组成的电压跟随器输出，供单片机检测。单片机检测到的电压V_{in}满足式(3.6)。

$$V_{in} = \mathrm{V_{out}} \frac{R_{19}}{R_{18} + R_{19}} \qquad (3.6)$$

故选取 R_{19}=15kΩ，R_{18}=10kΩ，使得V_{in}约为 3V，小于 3.3V 的范围限制。

3.4 输出电压与输出电流的单片机测量

输出电压和输出电流用单片机 TI TM4C1294NCPDT 的 ADC 模块进行测量，具体如下。

在电路连接上，将单片机 CH0/PE3 脚与稳压源板 J2 脚连接用于测量输出电压,将单片机 CH1/PE2 脚与稳压源板 J1 脚连接用于测量输出电流.。

在软件设计上，通过程序内的 ADCInit 函数使能采样序列产生器 1，分别用于测量输出电流和电压，并将其分别设为与采样通道 CH0/PE3 和 CH1/PE2 相连，从而实现对输出电压和电流的测量。

单片机程序设计流程图如下：

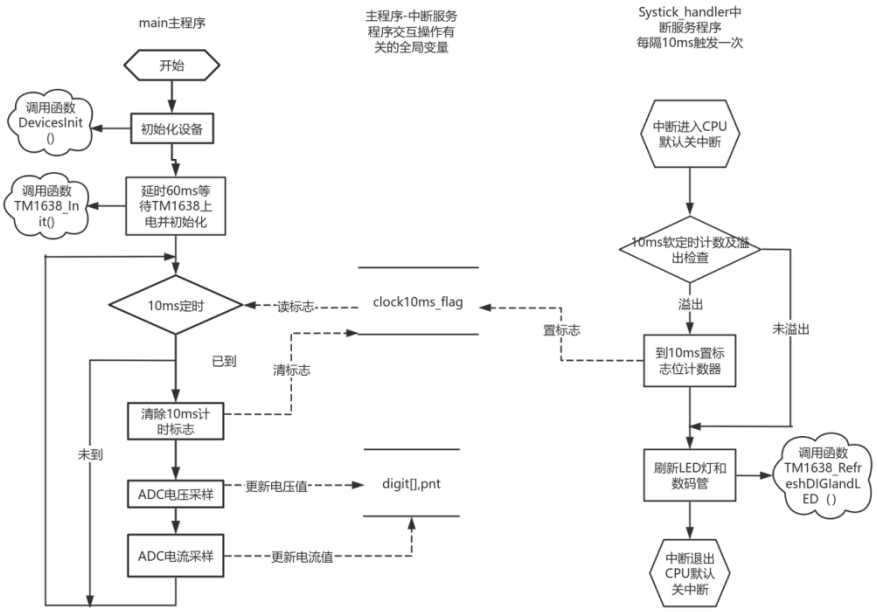


图 3.4 单片机程序流程图

如果直接将每次采样得的值直接送数码管显示，数码管显示值的跳变很快，不便读数，即显示的稳定度不够。于是，在进行电压采样与电流采样时考虑应用滑动平均算法:采用 10ms 的采样间隔，做连续 WINDOW_SIZE 个样值平均。取 WINDOW_SIZE=100，可以得到较稳定的显示值。

3.5 关键问题与解决

3.5.1 输出纹波[8]

BUCK 电路导致的纹波主要有以下两部分组分：

(1) 电路中开关交替充放电因素引起的波形成分通常表现为三角波。

(2) 开关切换瞬间的电磁涡流（在储能电感内表现最强烈）因素造成的波形成分往往为尖刺状。

故输出纹波大致形态为叠加尖刺状波形的三角波形，如图(3.5)所示；实际观测到的纹波往往很不规则，如图(3.6)所示。

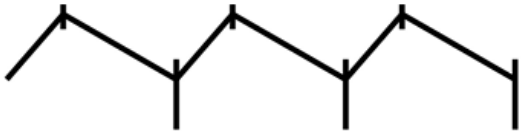


图 3.5 叠加尖刺状波形的三角波形

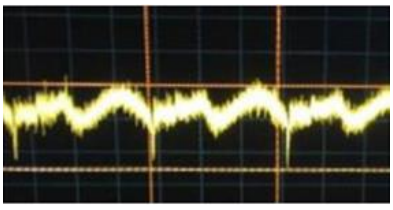


图 3.6 实际观测到的纹波

在实际过程中，示波器、探头及其他外界干扰所造成的尖刺状波形，会严重影响纹波的峰值形态。本组测量过程中，利用将探头倍数调整为 1:1、更换示波器探头、将探头夹取处设在水泥电阻引脚最短处等方法，成功获得了峰值较为合理的纹波。

3.5.2 电源转换效率

该电路的电源转换效率满足式(3.7)。

$$\eta = \frac{\frac{{V_{OUT}}^2}{R}}{V_{IN} \cdot I_{IN}} \qquad (3.7)$$

其中，V_{IN}、I_{IN}为电路整体输入的电压与电流，其数值可直接从稳压源显示屏上读取；V_{OUT}为负载R（即5.1Ω水泥电阻）两端的有效电压。

4. DC-DC 开关稳流供电系统的设计和实现

4.1 主要功能和设计指标

4.1.1 主要功能

本模块为基于上述稳压源改装的基于 TPS40200 芯片的 DC-DC 开关稳流电源模块。

该模块适配 10~20V 直流电压输入，可通过单片机程控进行 0.1~1A 稳流输出，并将输出电压和电流显示在数码管上。同时，该模块设置有过压保护功能，实现稳流源限压为 5V 左右。

4.1.2 设计指标

模块电路设计的技术指标如表 4.1 所示。

技术指标项	具体要求
输入电压	可适配直流 10~20V
输出电流	直流 0.1~1A 可调，以 0.1A 步进 开机默认 0.8A
输出限压值	5.2V±0.2V
电源转换效率	80%（合格）
输出过流/短路保护功能	
空载时，能按常规自动进入输出限压保护状态	

表 4.1 电路设计技术指标及要求

4.2 电路设计原理

DC-DC 开关稳流电源是基于 DC-DC 开关稳压电源改装而成，在稳压源的基础上加装了误差放大电路和限压保护电路。模块通过误差放大电路，将设定电流值和输出电流实际值的误差反馈至电

路，从而实现稳流输出。其中，设定电流值由单片机控制 DAC6571 的输出来调整。稳流源改装方案的电路设计图如图 4.1 所示：

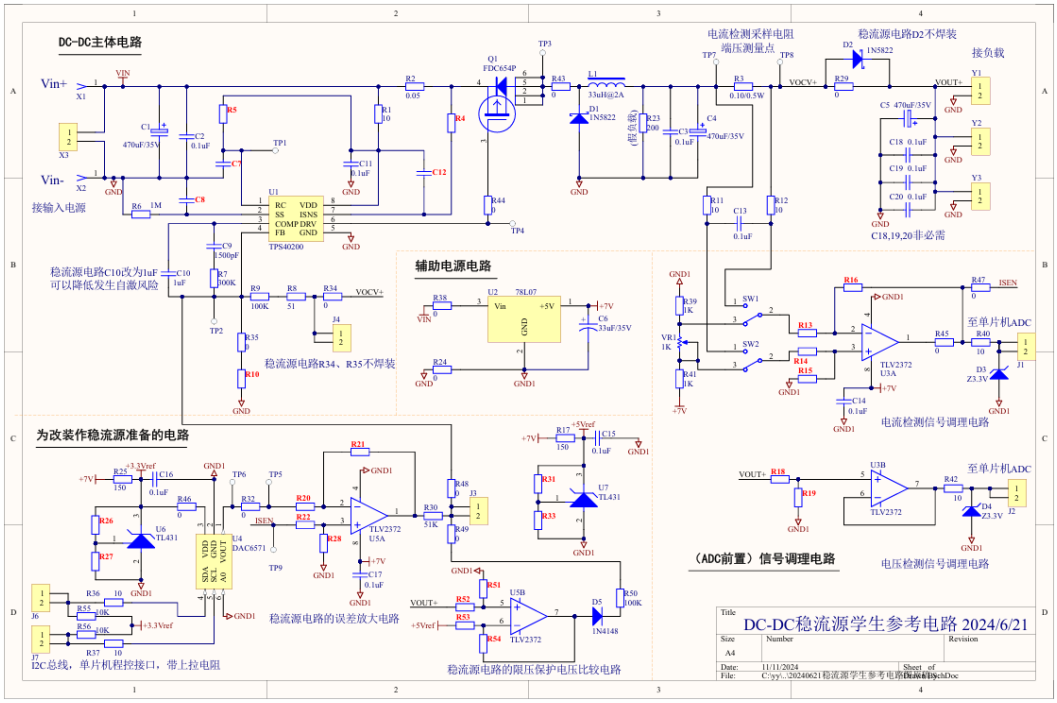


图 4.1 稳流源改装方案电路设计图[9]

单片机闭环程序控制框图如图 4.2 所示：

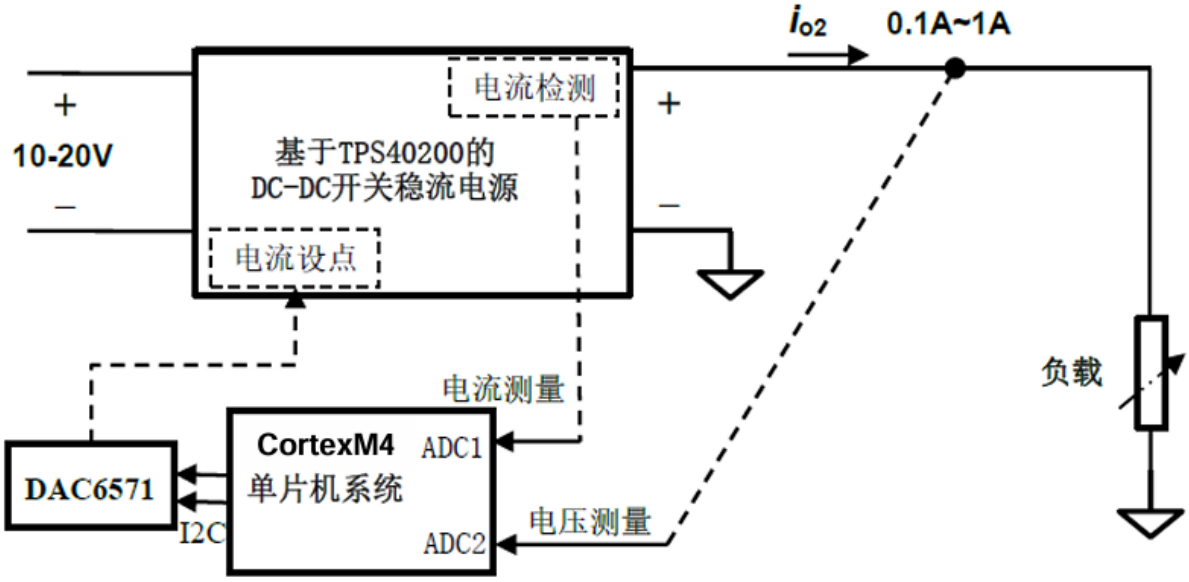


图 4.2 单片机闭环程控结构框图[10]

4.3 电路重要部分设计与参数确定

4.3.1 DAC6571 控制模块

DAC6571 芯片具有数模转换功能，其管脚功能描述如表 4.2 所示：

管脚标号	名称	描述
1	VOUT	输出 DAC 控制的模拟电压
2	GND	接地引脚
3	VDD	电源引脚（通常为 2.7~5.5V）
4	SDA	串行数据输入引脚
5	SCL	串行时钟输入引脚
6	A0	设备地址选择

表 4.2 DAC6571 芯片管脚描述[11]

工作时，芯片接收串行数据（SDA）、串行时钟信号（SCL）及模拟电压，按照时钟节拍将输入的编码数据转化为相应的模拟电压信号，最终输出电压值。在稳流源模块中围绕 DAC6571 设计的具体电路如图 4.3 所示：

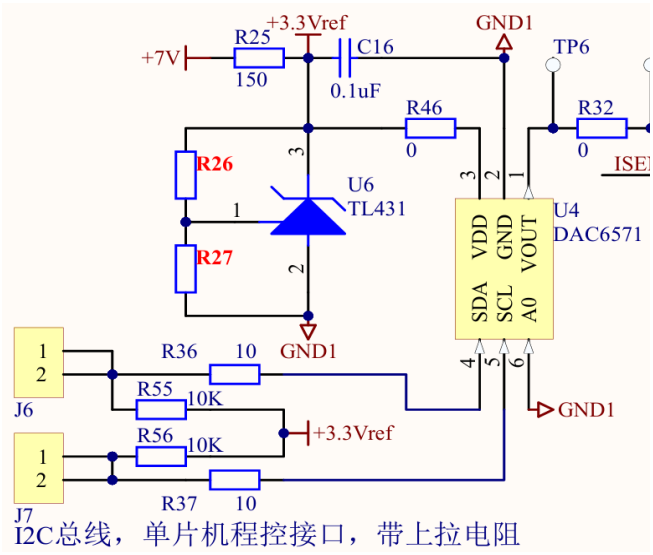


图 4.3 稳流源中 DAC6571 的相关电路图[12]

如图 4.3，DAC6571 芯片的 A0 和 GND 接地；VDD 接入围绕 TL431 的稳压管电路，以此获得稳定的工作电压，保证芯片正常工作；单片机通过 SDA 和 SCL 向芯片输入数字编码数据和时钟信号，完成数据的通信传输。

工作电压 VDD 由稳压管 TL431 及电阻 R26、R27 组成。TL431 产生 2.5V 的参考电压V_{ref}，则输出电压 VDD 大小如式(4.1)所示：

$$V_{DD} = \frac{R_{26} + R_{27}}{R_{27}} V_{ref} \qquad (4.1)$$

工作电压 VDD 取 3.3V，故取$R_{26}=16k\Omega$，$R_{27}=51k\Omega$。经过实验测量，该组电阻数值选择可以得到误差小于 5%的输出电压值。

4.3.2 误差放大电路

该电路核心为一个差分放大电路，作用是比较输出电流和设定电流，确定误差并放大。

误差放大电路的设计图如图 4.4 所示。

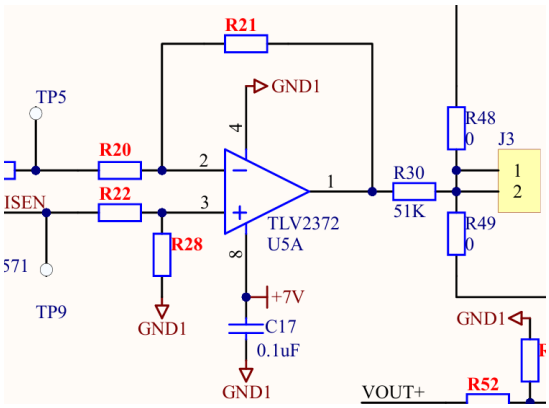


图 4.4 误差放大电路设计图[13]

TP5 为单片机设点的控制信号，ISEN 是检测到的输出电流信号。两信号的误差经过差分放大电路放大后输出至 TPS40200 芯片，以此调节输出电流，达到稳流输出的效果。

放大倍数k满足式(4.2)。设置放大倍数$k=10$，则选取$R_{20}=R_{22}=10k\Omega$，$R_{21}=R_{28}=100k\Omega$。

$$k=\frac{R_{28}}{R_{22}}=\frac{R_{21}}{R_{20}} \qquad (4.2)$$

4.3.3 限压保护电路

限压保护电路如图 4.5 所示，由一个差分放大电路和二极管组成，用于限制输出电压过大。VOUT+为监测的输出电压，+5Vref 为稳压管电路产生的基准电压。二极管 DA1 使电路的输出在正常状态下不影响误差放大电路的输出；而发生限压作用时，其输出对稳流放大造成限制。

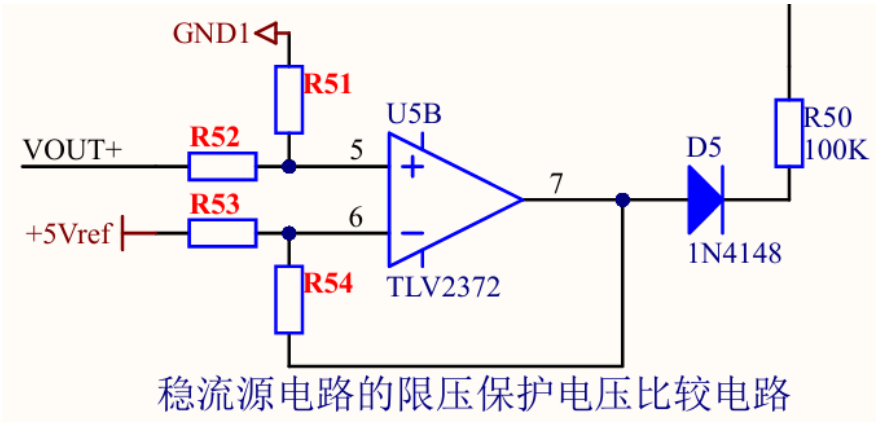


图 4.5 限压保护电路设计图[14]

放大倍数k满足式(4.3)。设置放大倍数$k=10$，则选取$R_{52}=R_{53}=10k\Omega$，$R_{51}=R_{54}=100k\Omega$。

$$k=\frac{R_{51}}{R_{52}}=\frac{R_{54}}{R_{53}} \qquad (4.3)$$

提供+5Vref的稳压管电路如图4.6所示。输入输出关系与式(4.1)类似，故取$R_{31}=R_{33}=10k\Omega$。经过实验测量，各项指标皆满足条件。

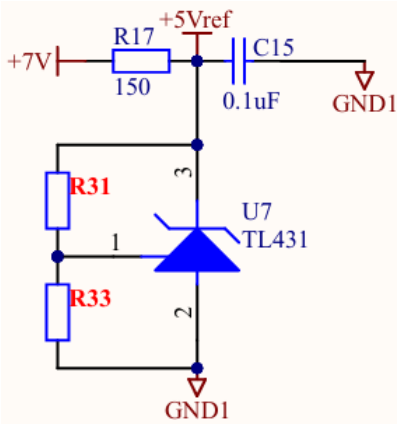


图 4.6 稳压管电路设计图[15]

4. 3. 4 其他电路修改

除了增添上述三个模块电路外，还需对原稳压源电路做一些改装。具体改装如图 4.7 所示：

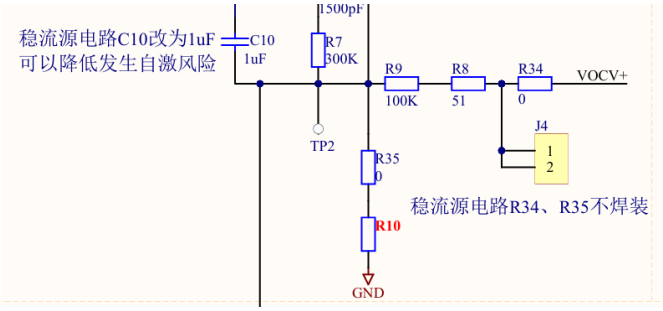


图 4.7 稳流源电路改装[16]

断开 R34、R35，将原电路与新增的电路模块连接起来；C10 由 10μF 换成 1μF，可以降低发生自激风险。

4. 4 标定和修正[17]

4. 4. 1 输出电流与其 ADC 转换编码之间的关系（关系式 A）

要实现单片机监测稳流源输出电流，需要确定被测的输出电流数值与 ADC 转换编码数值之间的关系式（以下简称关系式 A）。

为了得到多样化的原始被测信号数值，我们选择断开 R32 并使用外接信号源在 TP5 处提供直流电压信号，操控稳流源的输出电流。实验测量 R3 的两端电压数值，换算成输出电流数值；通过专

门编写的单片机程序，将对应的 ADC 测量值显示在实验底板数码管上；把两方面的成对数值列入表格中，然后选择数学方法拟合得到关系式 A。

考虑到不同负载的电流-ADC 转换关系不同可能影响测量的精确度，因此我们针对不同负载（3 欧、5.1 欧、10 欧）分别测量电流-ADC 转换关系。测量结果下列图表所示：

表 4.3 3 欧负载下输出电流与 ADC 测量值的对应关系

3 欧负载	ADC(A)	0.083	0.183	0.283	0.381	0.48	0.58	0.679	0.776	0.876	0.974
	实测电流(A)	0.1001	0.2006	0.3	0.4004	0.4997	0.609	0.7	0.7994	0.8993	0.9986

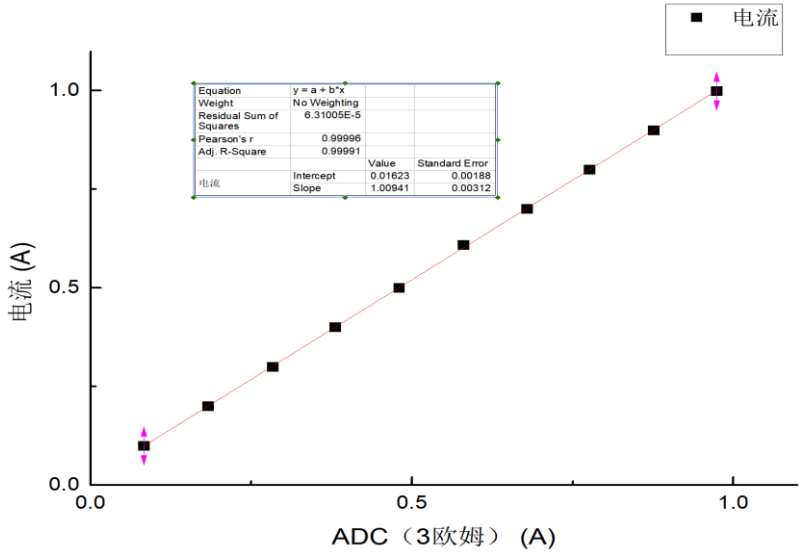


图 4.8 3 欧负载下电流-ADC 转换关系

表 4.4 5.1 欧负载下输出电流与 ADC 测量值的对应关系

5.1 欧负载	ADC(A)	0.084	0.182	0.279	0.378	0.475	0.574	0.671	0.77	0.867
	实测电流(A)	0.1003	0.2003	0.3008	0.4006	0.5003	0.6	0.6997	0.7996	0.8995

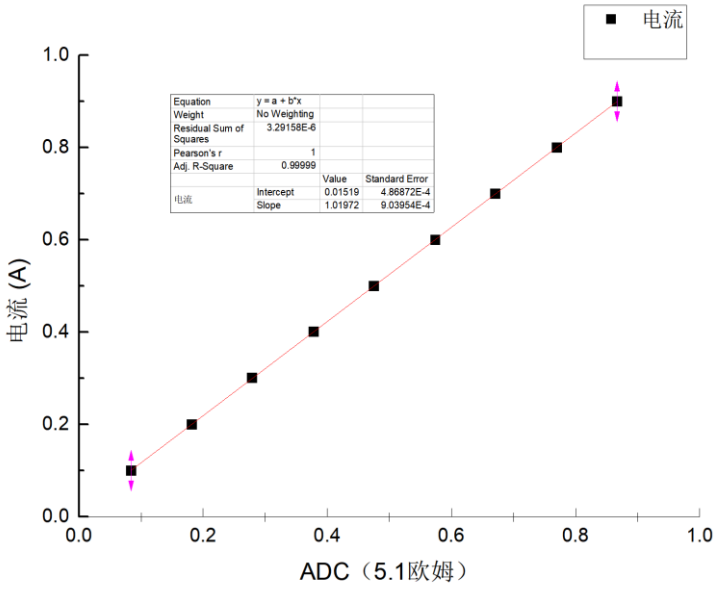


图 4.9 5.1 欧负载下电流-ADC 转换关系

表 4.5 10 欧负载下输出电流与 ADC 测量值的对应关系

10 欧负载	ADC(A)	0.083	0.18	0.273	0.369	0.464
	实测电流(A)	0.1001	0.2013	0.3002	0.3992	0.4997

由于输出电压超过 5V 触发限压保护，故不纳入计算。

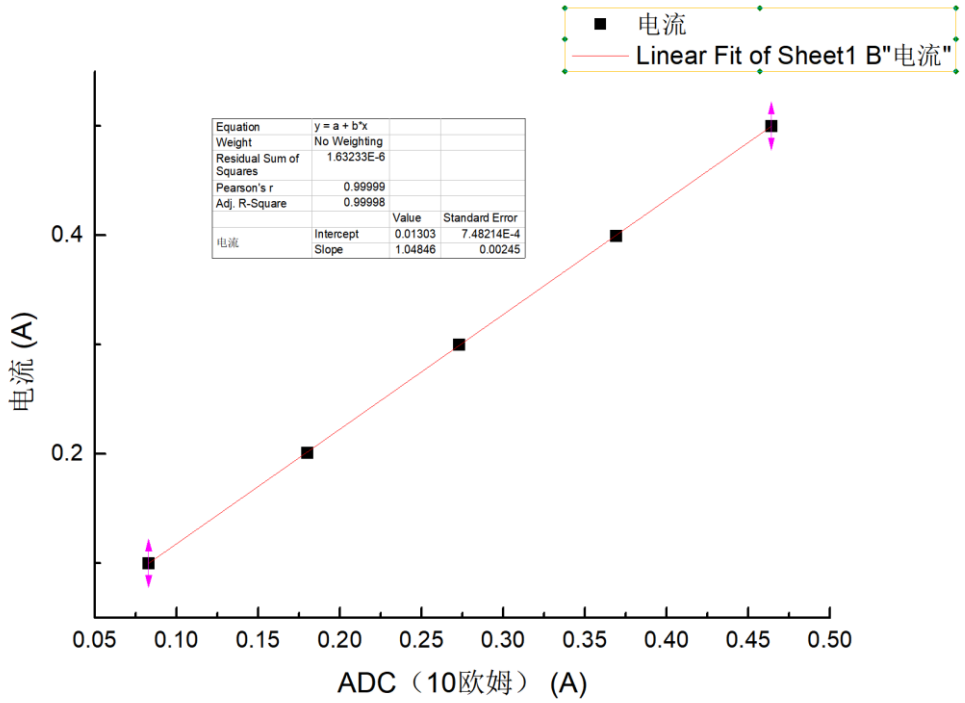


图 4.10 10 欧负载下电流-ADC 转换关系

4. 4. 2 输出电压与其 ADC 转换编码之间的关系（关系式 B）

由于输出电压与 ADC 转换编码在不同负载的情况下误差不大明显，因此并未对输出电压与 ADC 转换编码之间的关系进行细调。

4. 4. 3 DAC 编码与稳流设点之间的关系（关系式 C）

使用单片机控制稳流源电路的稳流设点，需要确定 DAC 编码数值与实际稳流设点数值之间的关系式（以下简称关系式 C）。

借用课程发布的 DAC6571 例程，通过编程使得通过按键能改变 DAC 编码数值。测量 R3 两端电压（并换算成输出电流读数），通过改变 DAC 编码使得实测电流尽可能接近电流设点，并记录下对应数据，如下表 4.6 所示：

表 4.6 DAC 编码与稳流设点的对应关系

实测电流(A)	0.1012	0.2022	0.2997	0.4007	0.5017	0.6022	0.7024	0.8008	0.9019
DAC 编码数值	4	38	69	101	133	165	197	228	260

根据表格通过线性拟合得到关系式 C，如图 4.11 所示：

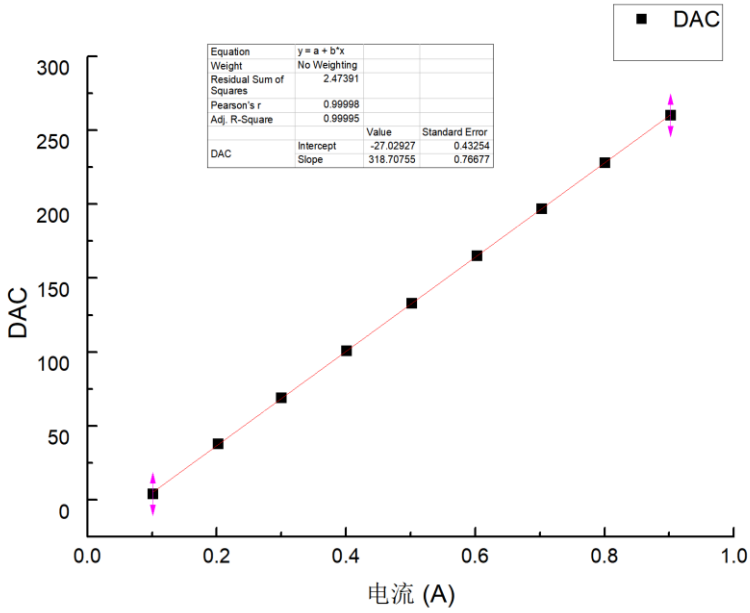


图 4.11 DAC 编码与稳流设点的对应关系式

4.5 单片机闭环控制式稳流算法

关于单片机闭环控制逻辑的状态转移图如图 4.12 所示：（3.4 节中的 ADC 算法仍然适用，在此不过多赘述）

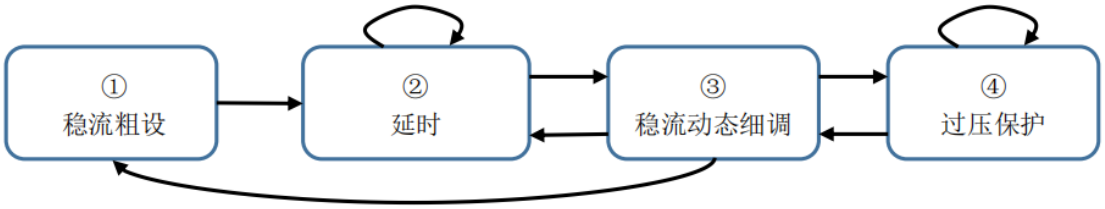


图 4.12 闭环控制逻辑的状态转移图

4.5.1 状态 1：稳流粗设

根据当前稳流设点目标数值$i_{设}$，利用关系式 C 求 DAC 编码M_{DAC}并发给 DAC 芯片。下一步转往状态 2。在状态 1 下，Systick_Handler 中断服务程序负责检测按键的状态，并根据按键输入调整稳流输出值。当主程序发现有按键被按下且设置了新的稳流值（即 DAC6571_flag==1 时，它会立刻通过数码管显示更新后的稳流值，并根据计算公式 C 得到的结果，生成相应的 DAC 编码并发送给 DAC 芯片；如果主程序检测到状态转换键被按下，则会立即准备跳转到状态 2。

具体的按键功能如表 4.7，上述逻辑对应的流程图如图 4.13。

表 4.7 稳流粗设的按键功能表

按键	功能

SW1	稳流值+100mA
SW4	稳流值-100mA
SW2	稳流值+10mA
SW5	稳流值-10mA
SW3	稳流值+1mA
SW6	稳流值-1mA
SW8	状态转换（下一步跳往状态 2）

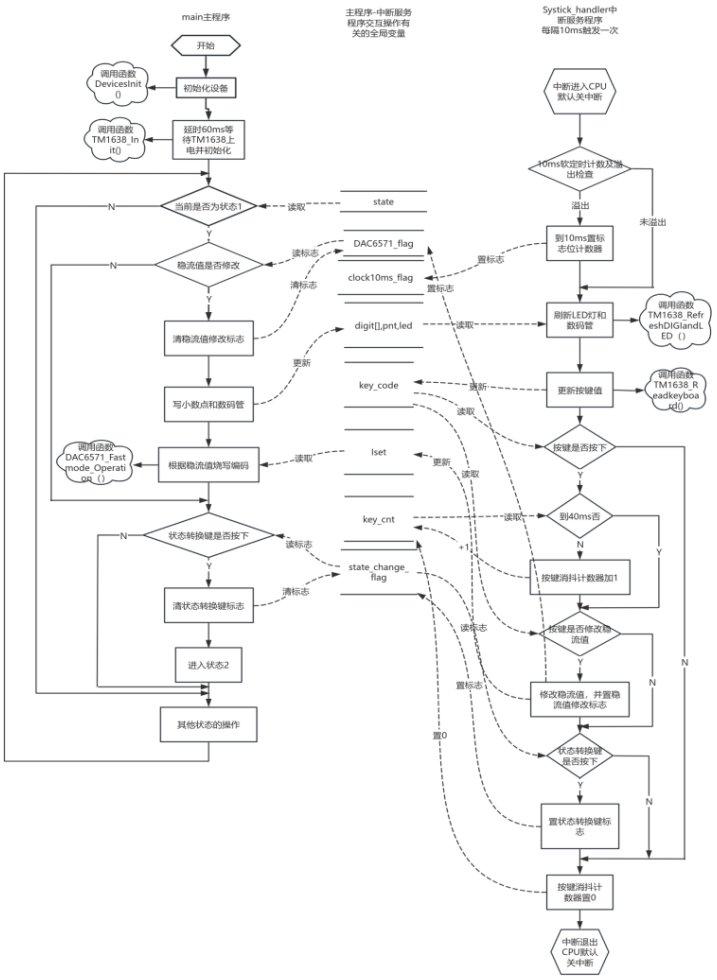


图 4.13 稳流粗设流程图

4.5.2 状态 2：延时

考虑到物理电路有反应时延，每次改变 DAC 编码设定后，应延时等待足够长时间。不过，此项等待时间也不宜太长，否则系统调整的速度节奏会太慢。因此，我们取 0.25s 延时，当延时达

成，下一步转往状态 3；否则停留在本状态。

4.5.3 状态 3：稳流动态细调

对输出电流做精细调控。根据当前稳流设点目标数值$i_{设}$、当前实际输出电流数值$i_{实}$、当前输出电压数值V_{out}，做出判断：

（1）若$i_{设}$发生变化，转往状态 1。

否则（2）若$i_{实} \geq i_{设}$，调整M_{DAC}，小幅降低电流设点控制值。下一步转往状态 2。

否则（3）若$i_{实} < i_{设}$ 且$V_{out} \leq 5.15(V)$调整M_{DAC}，小幅提高电流设点控制值。下一步转往状态 2。注 1：过压保护判断门限，可以根据自己电路特性灵活确定。

否则（4）$i_{实} < i_{设}$且$V_{out} > 5.15(V)$，判断为发生输出过压保护，暂停调整输出电流设点，转往状态 4。

上述逻辑对应的流程图如图 4.14 所示：

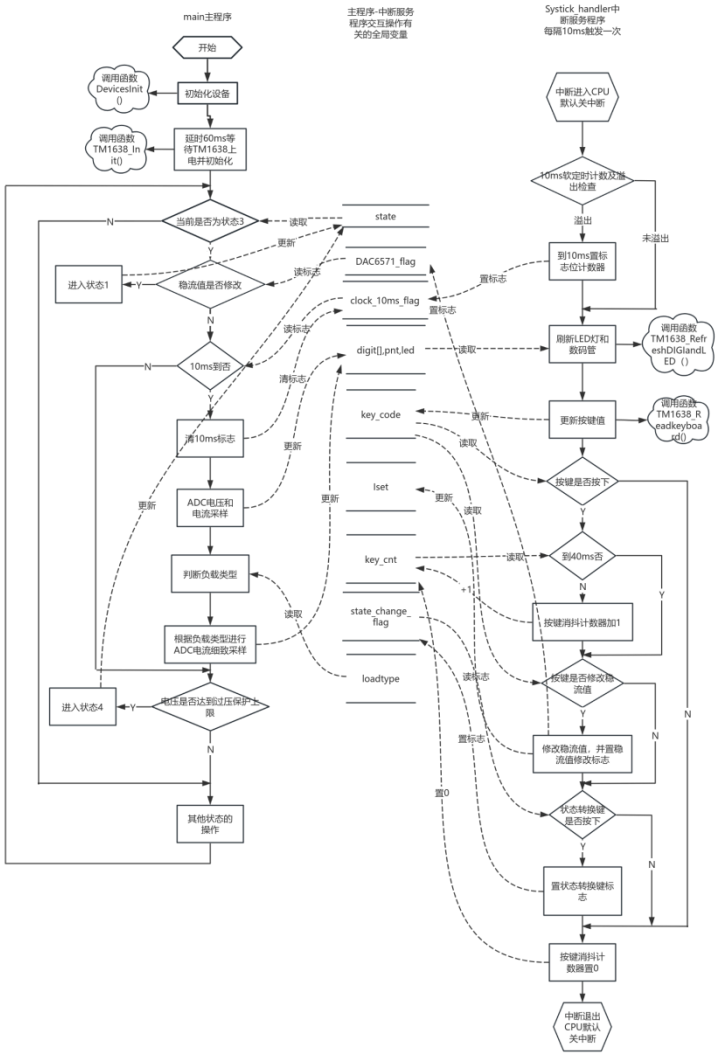


图 4.14 稳流动态细调流程图

4.5.4 状态 4：过压保护

因负载原因（比如处于空载状态）导致稳流源输出进入过压保护状态，等待负载状况发生变化。

（1）若$i_{实} < i_{设}$ 且 $V_{out} > 5.15(V)$，负载状况未变化，维持在状态 4；

否则（2）负载状况已变化，下一步转往状态 3。

4.6 关键问题解决

4.6.1 电源效率提高

初次测试时，稳流源电源转换效率仅有 80%左右。可知电源效率降低的主要因素有：

① 假负载 R23 消耗部分功率。

② 稳压管电路中的电阻 R15、R17 消耗较大功率。

因此，我们采取将 R15、R17 的取值提高至$1k\Omega$，将 R23 的取值提高至$2k\Omega$。经测量，电源转换效率有较明显的改善。

5. 开关稳流电源模块的 PCB 设计

5.1 稳流源电路原理图纸的绘制

5.1.1 工作步骤

在已经给出的稳压源参考电路图的基础上进行修改。添加二极管 D5 以及电阻 R48，R49，R50，R51，R52，R53，R54，R55，R56，用简明的走线将它们连接到原有的电路图上。确认所有元件和线条已经添加无误后可以在设计板块中选择 Update PCB Documents 来更新 PCB 图纸，为 PCB 图纸的绘制提供原理电路准备。

5.1.2 工作要领

首先要注意添加的二极管以及各个电阻元件的库的来源。二极管 D5 选取软件自带的库 Miscellaneous Devices.IntLib 中的 Diode 1N4148，电阻选取自己生成的集成库 PCBDCDC.IntLib 中的 0805 封装的贴片电阻。

其次要注意需要在封装管理器中修改一些元件的封装。如二极管 D5 需要找到 PCBDCDC.IntLib 库中的封装来执行封装变更。

改正一些元件的值使得电路取得更好的工作效果。将 C10 改为 1uF，可以降低发生自激的风险。

要时刻留意连线是否真的连上。三条线或更多条线的交汇处必定会出现一个点。若没有出现点，则说明实际上此处有线没有真正连上。

利用网络标签来方便地使原件与一些较远的端口连接。如这里添加了网络标签来使 R52 与 VOUT+连接。

原理图纸绘制过程中及绘制完成后需要及时保存。

5.2 稳流源电路 PCB 图纸的绘制

5.2.1 工作步骤

在使用 Update PCB Documents 对 PCB 图纸执行变更后，可以开始 PCB 图纸的绘制。首先需要删除掉多余的焊盘，然后需要删除掉原本图纸的下半部分的 Top Layer 和 Bottom Layer 的铺地。之后自己安排好 PCB 图纸上的元件布局，将新加入的元件添加到原 PCB 图纸的合理位置，之后绘制出相应的走线。在确认连线完成且无误后，重新进行 Top Layer 和 Bottom Layer 的铺地。最后，运

行 PCB 图纸的 DRC 检查，如果没有发现错误，则完成了对 PCB 图纸的绘制，可以进行后续的外发加工。

5.2.2 工作要领

注意焊盘删除，铺地删除以及走线删除时需要小心不要误删到周围的器件。

时刻注意自己是在哪一层上绘制 PCB 图纸。不要搞混 Top Layer，Bottom Layer 以及 TOP Overlay 这三个 PCB 层。

元件可以合理地灵活地旋转来使走线更加方便地连接，也使得连线看起来更加直观。

对于实在无法在顶层完成连接的走线，需要适当地添加过孔来在底层完成连线。

与电源不直接连接的一般的走线长度选定为 30mil 较为合适。

铺地的范围和参数需要与原来的 PCB 板图中的铺地的范围和参数保持完全一致，并且 NET 要选定为 GND,另外也一定要勾选 Pour Over All Same Net Objects，避免造成可能出现的原理性错误。

6. 并联供电系统及模块负载均衡的设计和实现

6.1 主要功能和设计指标

6.1.1 主要功能

并联供电系统是由上述稳压源、稳流源通过单片机程控实现并联均流输出。该系统适配10~20V 直流输入，可输出 5V 左右稳定直流电压，额定电流为 2A。单片机控制均流输出，即两模块输出电流基本相等。同时，单片机可监测输出电压、稳压源输出电流、稳流源输出电流，并显示在数码管上。

6.1.2 设计指标

模块电路设计的技术指标如表 6.1 所示。

技术指标项	具体要求
负载均衡度	负载大于额定负载 20%时，$\frac{\lvert 稳压源输出电流-稳流源输出电流\rvert}{输出电路\times 0.5}\leq 15\%$
单片机监测电压和电流	
均衡响应时间	负载突变到再次基本达到均衡态的时间在 2s 内
输出短路保护和恢复	输出能有效关断，未导致台式稳压电源过流告警，并能自动恢复正常输出

表 6.1 并联供电系统的设计指标[18]

6.2 电路设计原理

本阶段将上述稳压源和稳流源并联，达到均流输出的效果。电路设计如图 6.1 所示：

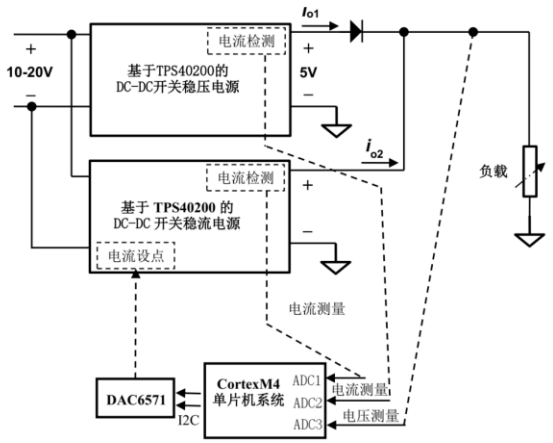


图 6.1 并联供电系统设计框图[19]

为了两模块并联适配，需要对硬件电路做一些改装[20]。

① R29 断开，加装 D2 二极管防止并联后外部电流倒灌，如图 6.2。

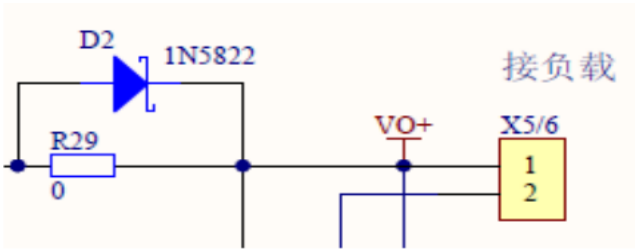


图 6.2 加装 D2 二极管示意图

② 调整 R10 取值为$15k\Omega$，将输出电压设点抬高至 5.3~5.4V，弥补通过上述二极管的压降损失，如图 6.3。

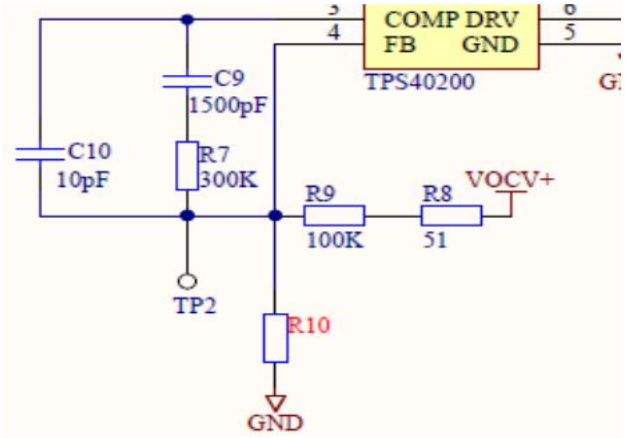


图 6.3 调整 R10 示意图

6.3 电路参数标定与修正

为了实现稳压源和稳流源的均流控制，我们首先需要确认在并联工作模式下，稳压源和稳流源的 ADC 测量是否依然准确。经过实验验证，稳压源的电流和电压测量，以及稳流源的电压测量误差都在可接受范围内。然而，在负载为 10 欧与 5.1//5.1 欧情况下，稳流源的电流测量误差超出了预期的范围。因此，我们重新对这些负载的稳流源的电流 ADC 测量关系式进行了标定。测得的数据如表 6.2、表 6.3、图 6.4、图 6.5 所示。

表 6.2 并联状态下稳流源 5.1//5.1 欧负载下电流-ADC 转换关系

5.1//5.1 欧负载	ADC(A)	0.093	0.192	0.289	0.387	0.482	0.586	0.686	0.783	0.879
	实测电流(A)	0.1004	0.2002	0.3004	0.4002	0.5005	0.6003	0.6998	0.8002	0.8997

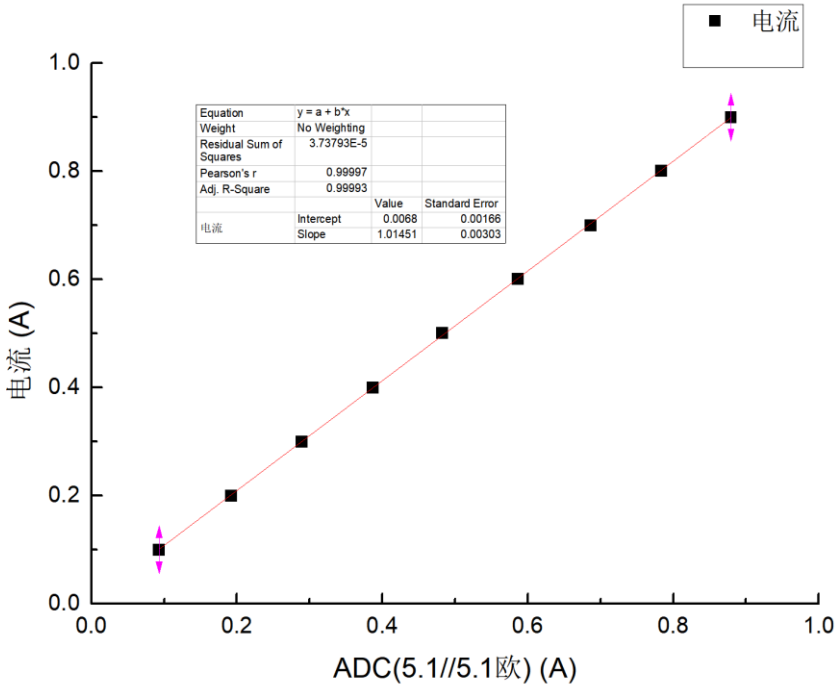


图 6.4 并联状态下稳流源 5.1//5.1 欧负载下电流-ADC 转换关系

表 6.3 并联状态下稳流源 10 欧负载下电流-ADC 转换关系

10 欧负载	ADC(A)	0.089	0.183	0.278	0.376	0.475
	实测电流(A)	0.1007	0.2002	0.3006	0.4003	0.4993

由于输出电压超过 5V 触发限压保护，故不纳入计算。

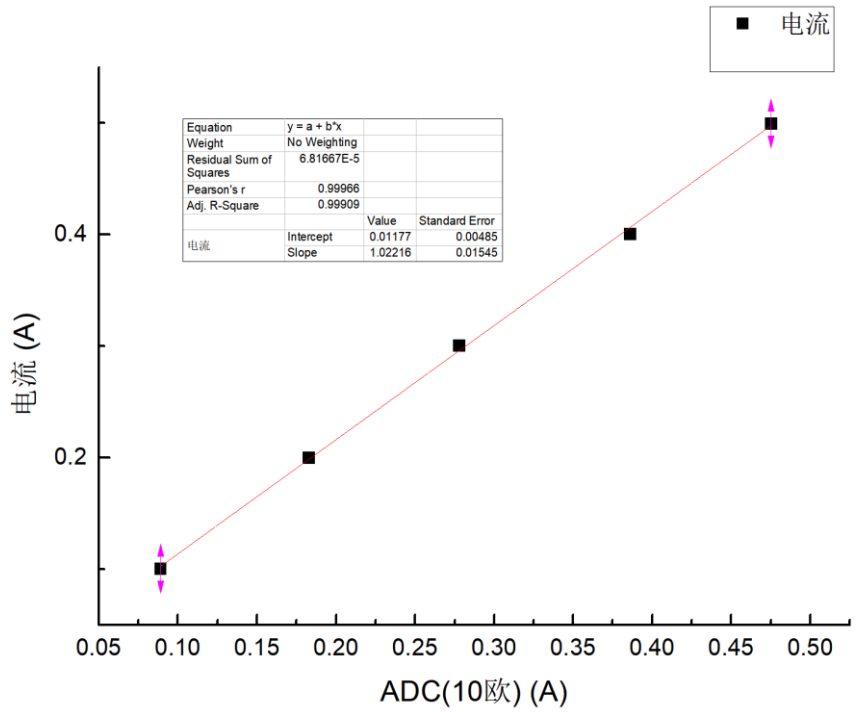


图 6.5 并联状态下稳流源 10 欧负载下电流-ADC 转换关系

6.4 单片机程控算法[21]

根据要求，单片机需要测量稳压源电流、稳流源电流和并联电压这三种信号，因此需要额外启用一个采样器。但由于数码管只有 8 个，因此必须通过切换显示来解决。我们将 SW8 设置为显示切换键，默认情况下，左侧的四个数码管显示稳压源电流，右侧的四个数码管显示稳流源电流。切换后，左侧四个数码管将显示并联电压。3.4 节中介绍的 ADC 测量算法依然适用，在此不再重复说明。此外，由于单片机只能控制稳流源的输出电流设定，每次发出新的控制命令，到系统有效响应，会有一段数十 ms 级的时延。达成稳定的动态均流效果的过程不能太久，一般负载突变后要在 4 秒内达

成，否则会影响使用者体验。因此，程控节奏不能太快也不能太慢，我们设定该时间间隔为 100ms，以确保系统能够稳定响应。

在阶段 3 中，还可能出现“跷跷板”现象使得系统无法进入稳定均流的正常工作状态，如图 6.6 所示。要避免“跷跷板”现象的发生，关键在于及时判断稳压源进入过流保护状态，并将稳流源的电流设定值调整到 1A（低于稳流源的过流保护阈值），以确保稳压源能够恢复正常输出。

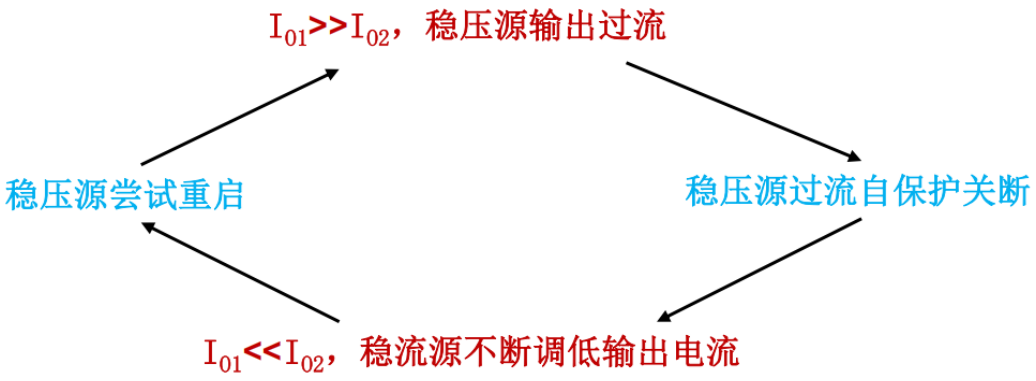


图 6.6 “跷跷板”现象

阶段 3 的闭环控制逻辑的状态转移图如图 6.7 所示：



图 6.7 阶段 3 的闭环控制逻辑的状态转移图

与 4.5 节的阶段 2 闭环控制逻辑基本相似（程序流程图也与阶段 2 相似，在此不过多赘述），其中做出的调整如下：

（1）稳流粗设阶段：稳流源的粗调电流设定由按键设置改为通过程序内部计算设定。单片机在状态 1 中通过 ADC 测量得到两路电流后，先取这两路电流的平均值，将其作为粗调电流值，然后将该值写入 DAC 芯片，并随后跳转到状态 2。

（2）延时阶段：将延时改为 100ms。

（3）稳流动态细调阶段：与阶段 2 类似，但针对稳流源电流与稳压源电流可能出现的差值过大问题进行了电流微调。

（4）过流保护阶段：将过压保护转换为过流保护。

7. 致谢

感谢所有课程组的老师们，感谢您们在整个学习过程中给予我们的悉心指导和耐心支持。通过在课前和课后提供详细的学习资料供我们自行查阅，再结合课堂上精心设计的讲解与答疑，老师们不仅帮助我们提高了自学能力，也让我们能够更好地理解和掌握实验中的关键知识。这种灵活多样的教学方法，不仅深化了我们的理论理解，还让我们在实践中得到了宝贵的锻炼，使我们得以在理论与实践之间架起了桥梁，真正做到了学以致用。

老师们在课堂之外留出的线下答疑时间以及在微信群里的及时答疑回复也对我们解决实验项目里遇到的困难和挑战起到了十分重要的作用。在答疑环节中，老师们为我们提供了宝贵的建议和支持，确保了我们能够顺利进行每一个实验任务。

感谢袁焱老师对整个课程各个实验环节的指导与关心以及对硬件板块的亲力指导与答疑，感谢孟桂娥老师对软件板块的亲力指导与答疑，感谢李安琪老师，杨晓老师对于 PCB 板块的亲力指导与答疑，感谢刘涛老师，诸勤敏老师认真的审查把控以及细心的答疑，也感谢所有老师在验收评测环节的耐心与体谅，感谢所有老师在整个课程任务开展过程中始终如一的专业精神，是你们的共同努力支持着我们成功完成这一门课程的所有项目与任务。再次致以敬意和感谢。

8. 参考文献

[1] 工程实践与科技创新[3A]讲座：第三阶段任务的技术要领

[2] 指导材料：宽输入非同步降压 DCDC 控制器芯片 TPS40200DATASHEET 导读

[3] 指导材料：宽输入非同步降压 DCDC 控制器芯片 TPS40200DATASHEET 导读

[4] 稳压源学生参考电路图

[5] 硬件技术实验 1_阻容元件功能通路的设计与测试

[6] 指导材料：宽输入非同步降压 DCDC 控制器芯片 TPS40200DATASHEET 导读

[7] 硬件技术实验 2_信号调理电路的设计与测试

[8] 工程实践与科技创新[3A]讲义_第一阶段项目任务的技术要领

[9] 20240621 稳流源学生参考电路图

[10] 工程实践与科技创新[3A]：第二阶段项目任务的技术要领

[11] DAC6571. http://www.ti.com.cn/product/cn/dac6571.

[12] 20240621 稳流源学生参考电路图

[13] 20240621 稳流源学生参考电路图

[14] 20240621 稳流源学生参考电路图

[15] 20240621 稳流源学生参考电路图

[16] 20240621 稳流源学生参考电路图

[17] 指导材料：第二阶段任务作品中程控方案的设计思路

[18] 工程实践与科技创新[3A]测试方法和评分规则

[19] 工程实践与科技创新[3A]讲座：第三阶段任务的技术要领

[20] 工程实践与科技创新[3A]讲座：第三阶段任务的技术要领

[21] 工程实践与科技创新[3A]讲座：第三阶段任务的技术要领

9. 附录 A　系统操作说明书

9.1 系统功能

本系统为 DC-DC 开关电源系统，适配 10~20V 直流电压输入，输出为 5V 左右直流稳定电压。本系统有稳压源模式、稳流源模式、单片机实施均流控制的并联式稳压模式。系统由单片机实施均流控制，实现了对两个电源模块的输出电流进行实时监控和动态精密均流调整。

9.2 使用方法

在稳压源的红黑端口接入直流电压，并联连接两模块的电压输入端口，并用杜邦线连接单片机和两模块。

9.3 稳压电源模式

系统通电后，按下按键 SW1 进入稳压电源模式，输入 10~20V 直流电压，输出 5V 直流电压，数码管左边显示输出电压，右边显示输出电流。该模块有限流保护功能。

9.4 稳流电源模式

在稳流电源模式中，输入 10~20V 直流电压，稳流源稳定输出 0.1~1A 电流。第一屏数码管右边显示为输出电流设点，此时通过按键可以调整输出电流，其中按下 SW1 输出电流+100mA，按下 SW4 输出电流-100mA。按下 SW8 确定电流设值。第二屏显示实测输出电压电流，数码管上左边显示输出电压，右边显示输出电流。该模块有限压保护功能，输出电压最大约为 5.2V。

9.5 并联式稳压供电模式

接通电源前，将两模块负载端用跳线连接，输入 10~20V 直流电压，输出 5V 直流电压，额定电流 2A。第一屏左边显示为稳压源输出电流，右边为稳流源输出电流，第二屏左边显示为并联输出电压。

10. 附录 B　测试和分析

10.1 测试项目和方法

表 10.1 DC-DC 稳压电源模块基本技术指标（卷面按 100 计，总评时折算）

检测项目	检测条件	检测方法	评分规则
输出电压绝对精度 (10)	R_{load}=5.1 Ω	测量输出电压	4.85~5.15V，得 5 分；其他得 0 分。
	R_{load}=∞（空载）		4.85~5.15V，得 5 分；其他得 0 分。
输出过流保护和恢复 (10)	输出 10.0V±0.1V	R_{load}=5.1 Ω//5.1 Ω；测量输出电压	输出能有效关断，得 5 分；其他得 0 分。
		Rload=5.1 Ω；再次测量输出电压	在上项有效关断基础上，能自动恢复正常输出，得 5 分；其他得 0 分。
负载调整率 (5)	输入 10.0V±0.1V	R_{load}=5.1 Ω和 10Ω时，分别测量输出电压	输出电压约为 5V，且变化小于 0.01V，得 5 分；大于等于 0.01V，小于等于 0.02V，得 2 分；其他，得 0 分。
电源转换效率 (40)	输入 10.0V±0.1V；R_{load}=5.1 Ω	测量输入电压、输入电流，输出电压（直接测量负载水泥电阻本体引脚部位，排除长导线的电阻的影响），负载以 5.1Ω计，计算效率。	若效率 η 大于 92%，得 40 分；η 在 80%至 92%之间，按公式计算得分（结果保留 1 位小数）；η 小于 80%，得 0 分。 $s = 40 - \frac{92\% - \eta}{12\%} \cdot 20$
电压调整率 (5)	R_{load}=5.1 Ω	输入从 10V 变化到 20V	输出电压约为 5V，且变化小于 0.01V，得 5 分；大于等于 0.01V，小于等于 0.025V，得 2 分；其他，得 0 分。
输出纹波 (20)	输入 20.0V±0.1V；R_{load}=5.1 Ω；示波器探头 X1 档，示波器带宽限制调为 20MHz	示波器（只用单通道，防止通道间干扰）观测交流纹波	若纹波 Upp 小于等于 30mV，得 20 分；在 30mV 至 100mV 间，按公式计算得分（结果保留 1 位小数）；超过 100mV 为不合格，得 0 分。 S=8+(100-U_{pp}）/70*12
违规使用电位器		目视检查	每违规使用一个电位器，扣6 分。
单片机监测电压和电流 (10)	输入 10.0V±0.1V；评测官任选一种工作状态（空载除外）进行检测	测量输出电压，与单片机显示数值对比	误差小于等于 0.02V，得 5 分；小于等于 0.05V，大于 0.02V，得 2 分；其他，得 0 分。
		测量输出电流，与单片机显示数值对比	误差小于等于 0.01A，得 5 分；小于等于 0.05A，大于 0.01A，得 2 分；其他，得 0 分。

表 10.2 DC-DC 稳流电源基本技术指标（卷面按100 计，总评时折算）

检测项目	检测条件	检测方法	评分规则
输出电流绝对精度(50)	输入 10.0V±0.1V；设定输出电流 0.1A 评测官指定 R_{load}=5.1 Ω或 10 Ω	人工设定输出电流时，只能观察UI信息，不得借助测量仪表的提示，且原则上更改设定在先，更换负载在后； 测量输出电流采样电阻端压； 测试过程不中断，不重新上电，不重启系统。	10±0.5mV 以内波动，得 10 分；10±1.0mV 以内波动，得 5 分；其他得 0 分。
	不下电改设定 0.2A，评测官指定 R_{load}=5.1 Ω或 10 Ω		20±0.5mV 以内波动，得 10 分；20±1.0mV 以内波动，得 5 分；其他得 0 分。
	不下电改设定 0.3A，评测官指定 R_{load}=5.1 Ω或 10 Ω		30±0.5mV 以内波动，得 10 分；30±1.0mV 以内波动，得 5 分；其他得 0 分。
	不下电改设定 0.5A，评测官指定 R_{load}=3 Ω或 5.1 Ω		49.0~51.0mV 波动，得 10 分；47.0~53.0mV 波动，得 5 分；其他得 0 分。
	不下电改设定 0.8A，评测官指定 R_{load}=3 Ω或 5.1 Ω		78.0~82.0mV 波动，得 10 分；74.0~86.0mV 波动，得 5 分；其他得 0 分。
输出限压保护和恢复(15)	输入 10.0V±0.1V；设定输出电流 1.0A	R_{load}=10 Ω 测量输出电压	限压输出 5.0~5.4V，得 5 分；其他得 0 分。
		不下电改为空载测量输出电压	限压输出 5.0~5.4V，得 5 分；其他得 0 分。
		不下电更换 R_{load}=3 Ω测量输出电流采样电阻端压	输出电流恢复正常，得 5 分；其他得 0 分。

电源转换效率 (20)	输入 10.0V±0.1V；R_{load}=5.1 Ω； 设定输出电流 1.0A	测量输入电压、输入 电流，输出电压（直接 测量负载水泥电阻本 体引脚部位，排除长 导线的电阻的影响）， 负载以 5.1Ω计，计算 效率。	若效率 η大于 90%，得 20 分； η在80%至 90%之间，按公式计算得分（结果保留 1 位小数）；η小于 80%，得 0 分。 $s = 20 - \frac{90\% - \eta}{10\%} \cdot 8$
印刷电路板	印刷电路板必须是定制加工，并在丝印或布线层带本组（组号/姓名）标记	目视检查	使用非定制加工电路板，扣 20 分； 定制加工电路板无本组或非本组标记，扣 15 分； 每违规使用一个电位器，扣6 分。
单片机监测电压和电流 (15)	输入 10.0V±0.1V； 评测官任选一种输出≥0.5A 的工作状态进行检测	测量输出电压，与单片机显示数值对比	误差小于等于 0.02V，得 7 分； 小于等于 0.05V，大于 0.02V，得 3 分；其他，得 0 分。
		测量输出电流，与单片机显示数值对比	误差小于等于 0.01A，得 8 分； 小于等于 0.05A，大于 0.01A，得 4 分；其他，得 0 分。

表 10.3 并联输出基本技术指标（卷面满分100，总评时折算）

检测项目	检测条件	检测方法	评分规则
负载均衡度(60)	输入 10.0V±0.1V；R_{load}=10 Ω	测量并联输出电压；测量两路输出电流采样电阻端压	三种检测条件下分别评测，各按以下规则： 并联输出电压须稳定在 4.85~5.15V，否则不得分。 第一档：负载均衡误差[注1]小于等于 5%，且两路电阻端压各自波动均小于等于 2mV，得 20 分； 第二档：不符合第一档，但负载均衡误差小于等于 10%，且两路电阻端压各自波动均小于等于 8mV，得 10 分； 第三档：不符合第一、二档，但负载均衡误差小于等于 15%，且两路电阻端压各自波动均小于等于 24mV，得 5 分； 以上均不满足，得 0 分。其他缺憾酌情扣分。
	不下电更换 R_{load}=5.1 Ω		
	不下电更换 R_{load} 由评测官指定（输出不超过额定电流）		
单片机监测电压和电流(15)	输入 10.0V±0.1V；评测官任选一种工作状态（输出总电流大于 0.8A）进行检测	测量输出电压，与单片机显示数值对比	误差小于等于 0.02V，得 5 分； 小于等于 0.05V，大于 0.02V，得 2 分；其他，得 0 分。
		测量第一路输出电流，与单片机显示数值对比	误差小于等于 0.01A，得 5 分； 小于等于 0.05A，大于 0.01A，得 2 分；其他，得 0 分。
		测量第二路输出电流，与单片机显示数值对比	误差小于等于 0.01A，得 5 分； 小于等于 0.05A，大于 0.01A，得 2 分；其他，得 0 分。
示波器测定均衡响应时间(10)	初始 R_{load}=5.1 Ω，基本达到均衡态；不下电更换 R_{load}=5.1 Ω //5.1 Ω，测量从负载突变到再次基本达到均衡态的时间	使用正确的技巧操作示波器	本项主要考核仪器操作技能，但必须使用本组自有作品；均衡度性能略有缺陷，原则上不影响本项得分。但是，若作品完全无法提供本项测试条件的，则不可得分。 能正确完成检测操作，得 10 分；否则根据缺憾程度，酌情得分/扣分。
输出短路保护和恢复(15)	输入 10.0V±0.1V	R_{load}=0 Ω（导线短路）	输出能有效关断，未导致台式稳压电源过流告警，得 5 分；其他得 0 分。
		R_{load}=5.1 Ω；再次测量输出电压	在上项有效关断基础上，能自动恢复正常输出，得 10 分；其他得 0 分。

[注 1] 负载均衡误差计算公式

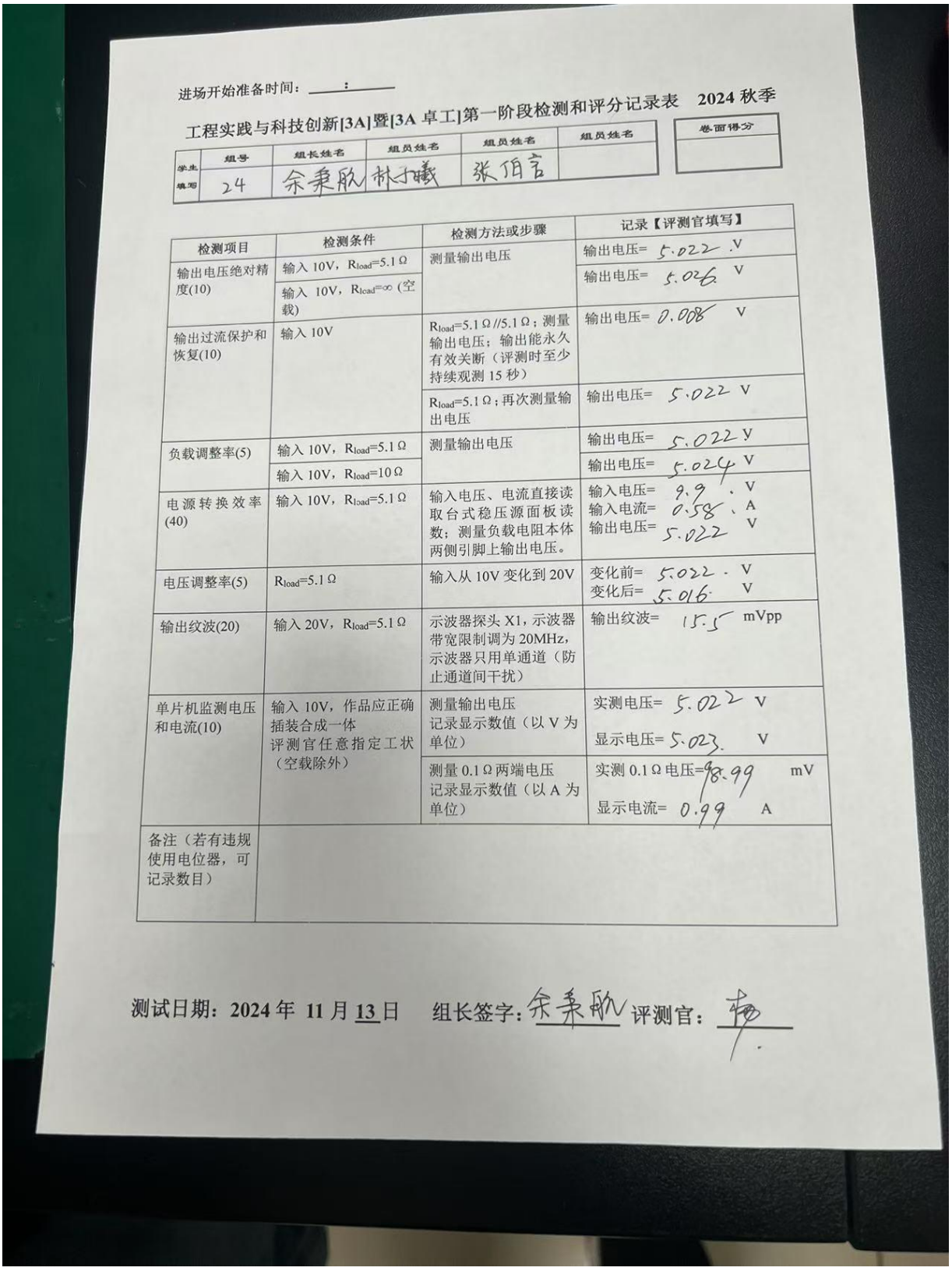
$$\frac{|\text{稳压源输出电流} - \text{稳流源输出电流}|}{\text{输出总电流} \times 0.5} \times 100\%$$

10.2 测试的资源

测试资源：实验台、万用表、稳压电源、电脑、示波器、待测稳压源系统、引线若干。

10.3 测试结果及分析

第一阶段



进场开始准备时间：____：____

工程实践与科技创新[3A]暨[3A 卓工]第一阶段检测和评分记录表 2024 秋季

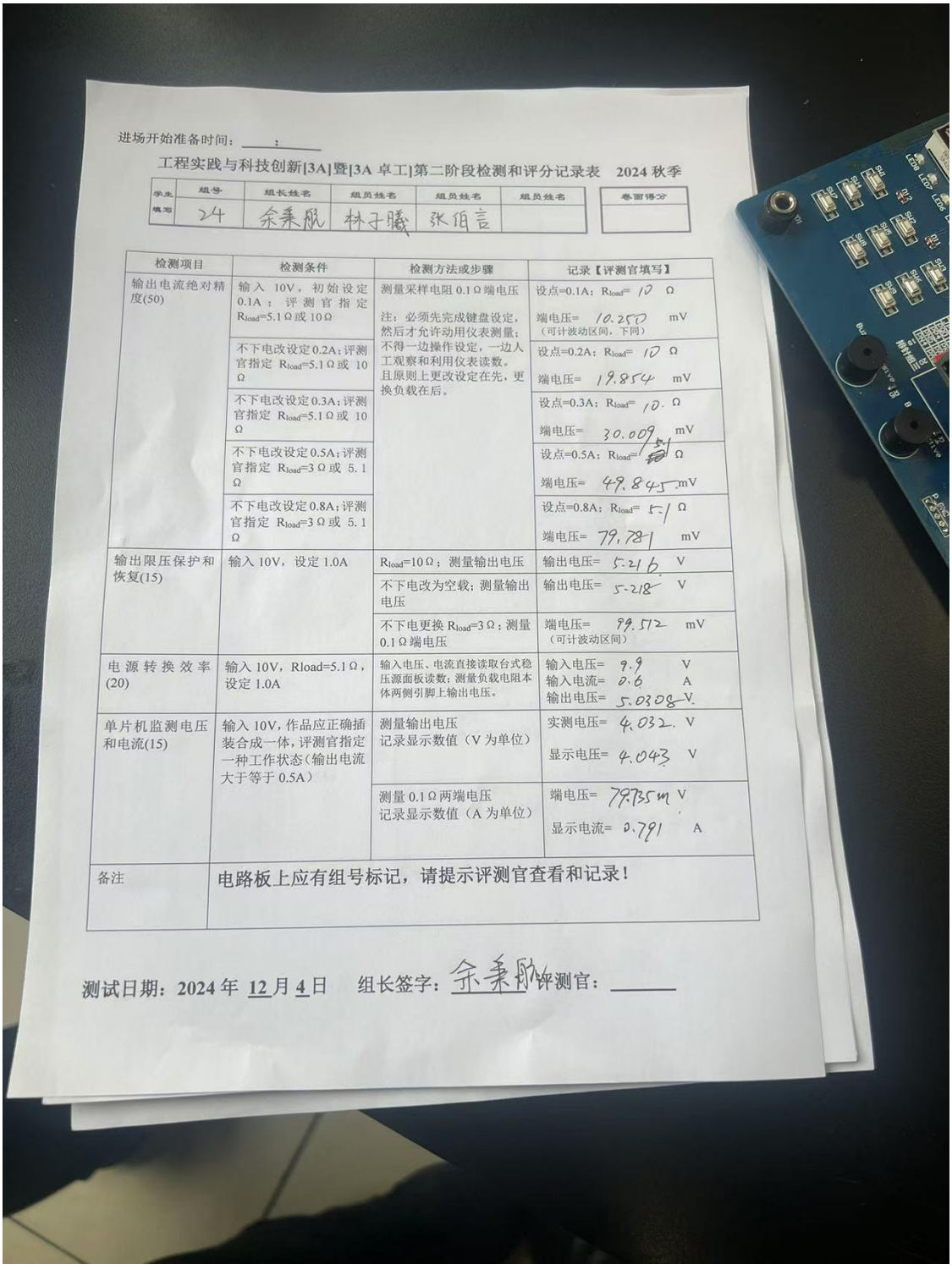
学生填写	组号	组长姓名	组员姓名	组员姓名	组员姓名	卷面得分
	24	余秉航	林子曦	张顶言		

检测项目	检测条件	检测方法或步骤	记录【评测官填写】
输出电压绝对精度(10)	输入 10V，R_{load}=5.1 Ω	测量输出电压	输出电压= 5.022 V
	输入 10V，R_{load}=∞ (空载)		输出电压= 5.026 V
输出过流保护和恢复(10)	输入 10V	R_{load}=5.1 Ω //5.1 Ω；测量输出电压；输出能永久有效关断（评测时至少持续观测 15 秒）	输出电压= 0.008 V
		R_{load}=5.1 Ω；再次测量输出电压	输出电压= 5.022 V
负载调整率(5)	输入 10V，R_{load}=5.1 Ω	测量输出电压	输出电压= 5.022 V
	输入 10V，R_{load}=10 Ω		输出电压= 5.024 V
电源转换效率(40)	输入 10V，R_{load}=5.1 Ω	输入电压、电流直接读取台式稳压源面板读数；测量负载电阻本体两侧引脚上输出电压。	输入电压= 9.9 V 输入电流= 0.58 A 输出电压= 5.022 V
电压调整率(5)	R_{load}=5.1 Ω	输入从 10V 变化到 20V	变化前= 5.022 V 变化后= 5.016 V
输出纹波(20)	输入 20V，R_{load}=5.1 Ω	示波器探头 X1，示波器带宽限制调为 20MHz，示波器只用单通道（防止通道间干扰）	输出纹波= 15.5 mVpp
单片机监测电压和电流(10)	输入 10V，作品应正确插装合成一体 评测官任意指定工状（空载除外）	测量输出电压 记录显示数值（以 V 为单位）	实测电压= 5.022 V 显示电压= 5.023 V
		测量 0.1 Ω 两端电压 记录显示数值（以 A 为单位）	实测 0.1 Ω 电压= 98.99 mV 显示电流= 0.99 A
备注（若有违规使用电位器，可记录数目）			

测试日期：2024 年 11 月 13 日　组长签字：余秉航　评测官：

图 10.1 第一阶段检测记录表

第二阶段



进场开始准备时间：____：____

工程实践与科技创新[3A]暨[3A 卓工]第二阶段检测和评分记录表 2024 秋季

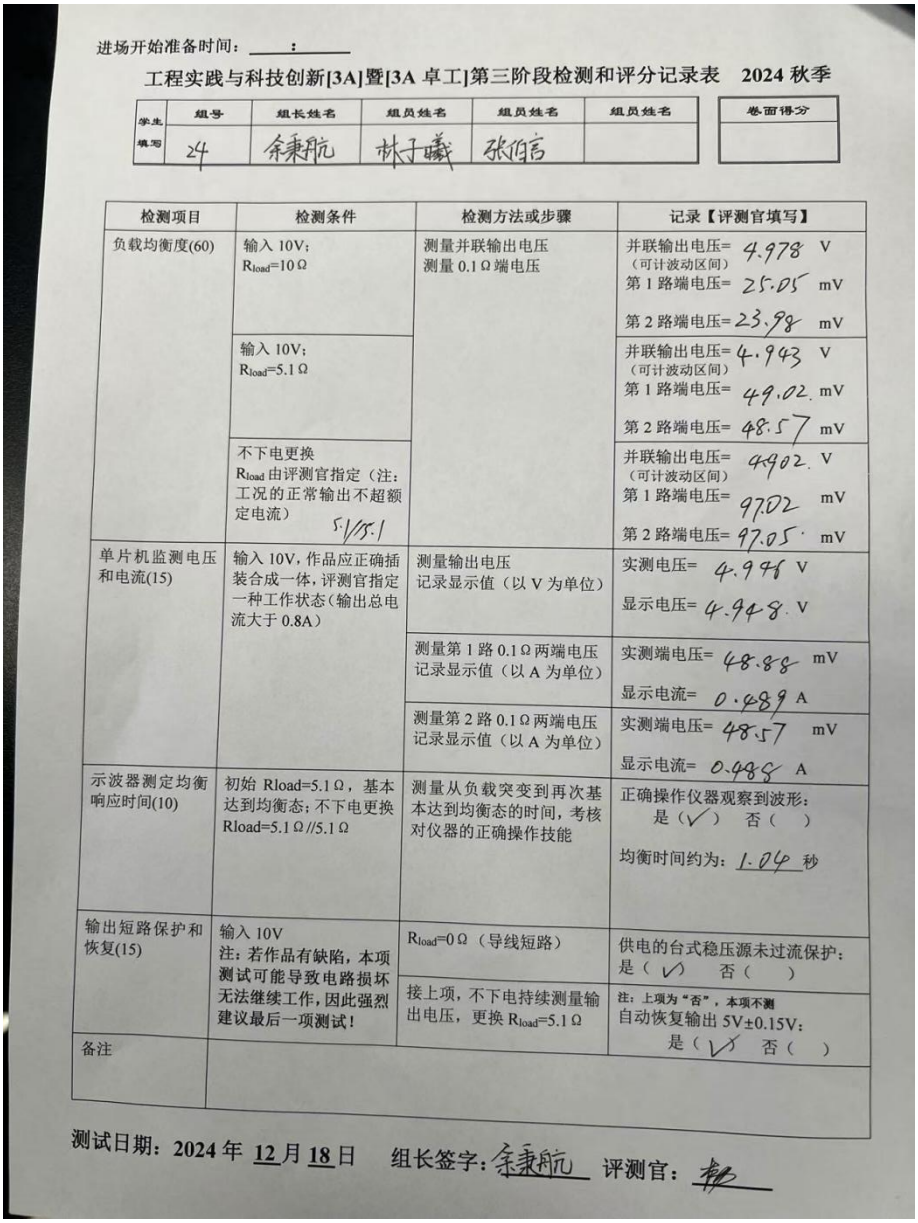
学生填写	组号	组长姓名	组员姓名	组员姓名	组员姓名	卷面得分
	24	余秉枫	林子曦	张佰言		

检测项目	检测条件	检测方法或步骤	记录【评测官填写】
输出电流绝对精度(50)	输入 10V，初始设定 0.1A；评测官指定 R_{load}=5.1Ω或 10Ω	测量采样电阻 0.1Ω端电压 注：必须先完成键盘设定，然后才允许动用仪表测量；不得一边操作设定，一边人工观察和利用仪表读数。且原则上更改设定在先，更换负载在后。	设点=0.1A；R_{load}= 10 Ω 端电压= 10.250 mV （可计波动区间，下同）
	不下电改设定 0.2A；评测官指定 R_{load}=5.1Ω或 10Ω		设点=0.2A；R_{load}= 10 Ω 端电压= 19.854 mV
	不下电改设定 0.3A；评测官指定 R_{load}=5.1Ω或 10Ω		设点=0.3A；R_{load}= 10. Ω 端电压= 30.009 mV
	不下电改设定 0.5A；评测官指定 R_{load}=3Ω或 5.1Ω		设点=0.5A；R_{load}= Ω 端电压= 49.845 mV
	不下电改设定 0.8A；评测官指定 R_{load}=3Ω或 5.1Ω		设点=0.8A；R_{load}= 5.1 Ω 端电压= 79.781 mV
输出限压保护和恢复(15)	输入 10V，设定 1.0A	R_{load}=10Ω；测量输出电压	输出电压= 5.216 V
		不下电改为空载；测量输出电压	输出电压= 5.218 V
		不下电更换 R_{load}=3Ω；测量 0.1Ω端电压	端电压= 99.572 mV （可计波动区间）
电源转换效率(20)	输入 10V，Rload=5.1Ω，设定 1.0A	输入电压、电流直接读取台式稳压源面板读数；测量负载电阻本体两侧引脚上输出电压。	输入电压= 9.9 V 输入电流= 0.6 A 输出电压= 5.0308 V
单片机监测电压和电流(15)	输入 10V，作品应正确插装合成一体，评测官指定一种工作状态（输出电流大于等于 0.5A）	测量输出电压 记录显示数值（V 为单位）	实测电压= 4.032 V 显示电压= 4.043 V
		测量 0.1Ω两端电压 记录显示数值（A 为单位）	端电压= 79.735m V 显示电流= 0.791 A
备注	电路板上应有组号标记，请提示评测官查看和记录！		

测试日期：2024 年 12 月 4 日 组长签字：余秉枫 评测官：_____

图 10.2 第二阶段检测记录表

第三阶段



进场开始准备时间：　　：

工程实践与科技创新[3A]暨[3A 卓工]第三阶段检测和评分记录表　2024 秋季

学生填写	组号	组长姓名	组员姓名	组员姓名	组员姓名	卷面得分
	24	余秉航	林子曦	张伯言		

检测项目	检测条件	检测方法或步骤	记录【评测官填写】
负载均衡度(60)	输入 10V；R_{load}=10 Ω	测量并联输出电压 测量 0.1 Ω 端电压	并联输出电压= 4.978 V（可计波动区间） 第 1 路端电压= 25.05 mV 第 2 路端电压= 23.98 mV
	输入 10V；R_{load}=5.1 Ω		并联输出电压= 4.943 V（可计波动区间） 第 1 路端电压= 49.02 mV 第 2 路端电压= 48.57 mV
	不下电更换 R_{load} 由评测官指定（注：工况的正常输出不超额定电流） 5.1/5.1		并联输出电压= 4902 V（可计波动区间） 第 1 路端电压= 97.02 mV 第 2 路端电压= 97.05 mV
单片机监测电压和电流(15)	输入 10V，作品应正确插装合成一体，评测官指定一种工作状态（输出总电流大于 0.8A）	测量输出电压 记录显示值（以 V 为单位）	实测电压= 4.948 V 显示电压= 4.948 V
		测量第 1 路 0.1 Ω 两端电压 记录显示值（以 A 为单位）	实测端电压= 48.88 mV 显示电流= 0.489 A
		测量第 2 路 0.1 Ω 两端电压 记录显示值（以 A 为单位）	实测端电压= 48.57 mV 显示电流= 0.485 A
示波器测定均衡响应时间(10)	初始 Rload=5.1 Ω，基本达到均衡态；不下电更换 Rload=5.1 Ω//5.1 Ω	测量从负载突变到再次基本达到均衡态的时间，考核对仪器的正确操作技能	正确操作仪器观察到波形： 是（√）　否（　） 均衡时间约为：1.04 秒
输出短路保护和恢复(15)	输入 10V 注：若作品有缺陷，本项测试可能导致电路损坏无法继续工作，因此强烈建议最后一项测试！	R_{load}=0 Ω（导线短路）	供电的台式稳压源未过流保护： 是（√）　否（　）
		接上项，不下电持续测量输出电压，更换 R_{load}=5.1 Ω	注：上项为“否”，本项不测 自动恢复输出 5V±0.15V： 是（√）　否（　）
备注			

测试日期：2024 年 12 月 18 日　组长签字：余秉航　评测官：

图 10.3　第三阶段检测记录表

11. 附录 C　课程学习心得和意见建议

张伯言：通过这次课程的学习，我绘制并外发加工得到了两块 PCB 电路板，初步学习了软件 Altium Designer 的使用方法，掌握了元件布局、布线、设计规则检查（DRC）等技术，较为完整地体验了设计简单的 PCB 板所需要经历的全部步骤。我在设计的过程中也犯过一些小错误，遇到过一些困难，不过通过自己解决以及请教老师最终都得以解决。总的来说，这次有关 PCB 设计的课程项目经历让我有了参与一段简单工程项目的体验，锻炼了我处理实际工程问题的能力，也让我对电子设计相关的领域有了更多的了解。

林子曦：这次实验增强了我们的动手能力与团队协作能力，使我们受益匪浅。感谢课程组老师们在网上提供详细的学习资料，供我们自学，更好地完成实验任务。同时也感谢老师们在课程微信群里的答疑。特别感谢袁焱老师在实验室的答疑，甚至还亲自为同学排查错误。最后感谢小组成员的全力配合使得我们此次实验圆满成功。

余秉航：这次实验，我负责了硬件的部分，在队友的协作下顺利完成了目标。我不仅对电子方向的理论知识和实验实践有了更深入的体会，更是学习到了团队协作的魅力。感谢队员和老师的帮助，我们收获了一个满意的结果。

12. 附录 D　软件程序清单

第一阶段

```
#include <stdint.h>
#include <stdbool.h>
#include "inc/hw_memmap.h"
#include "inc/hw_types.h"
#include "driverlib/debug.h"
#include "driverlib/gpio.h"
#include "driverlib/pin_map.h"
#include "driverlib/sysctl.h"
#include "driverlib/systick.h"
#include "driverlib/interrupt.h"
 #include "driverlib/adc.h"
#include "tm1638.h"

#define SYSTICK_FREQUENCY                100
#define V_T10ms          1
#define V_T40ms 4
#define V_T100ms 10
#define WINDOW_SIZE 100
void GPIOInit(void);
void ADCInit(void);
void SysTickInit(void);
void DevicesInit(void);
void ADC_Sample(void);
void ADCHardwareOversampleConfigure(uint32_t ui32Base,uint32_t ui32Factor);
```

```
uint8_t clock40ms = 0;
uint8_t clock100ms = 0;
uint8_t clock10ms = 0;
uint8_t clock40ms_flag = 0;
uint8_t  clock100ms_flag = 0;
uint8_t clock10ms_flag = 0;
uint8_t digit[8]={' ',' ',' ',' ',' ',' ',' ',' '};
uint8_t pnt = 0x11;
uint8_t led[] = {0, 0, 0, 0, 0, 0, 0, 0};

uint32_t ui32SysClock;

uint32_t pui32ADC0Value[4];
uint32_t samples =0;
 uint32_t datas[WINDOW_SIZE] = {0};
uint32_t datas1[WINDOW_SIZE] = {0};
uint32_t sum = 0;
uint32_t sum1 = 0;
uint32_t average = 0;
uint32_t average1 = 0;
uint32_t x = 0;
uint32_t x1 = 0;
 int main(void)
{
        DevicesInit();
        while (clock100ms < 3);
        TM1638_Init();
```

```
    while (1)
    {
if (clock10ms_flag == 1)
{

GPIOPinWrite(GPIO_PORTC_BASE, GPIO_PIN_7, GPIO_PIN_7);
                        ADC_Sample();
GPIOPinWrite(GPIO_PORTC_BASE, GPIO_PIN_7, 0x0);
pui32ADC0Value[0] =(uint32_t)pui32ADC0Value[0]*3300*5/3/4095;
pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300*10/15/4095;

sum += pui32ADC0Value[0];
sum -= datas[x];
datas[x] = pui32ADC0Value[0];
x = (x+1)%WINDOW_SIZE;
average = sum / WINDOW_SIZE;
sum1 += pui32ADC0Value[1];
sum1 -= datas1[x1];
datas1[x1] = pui32ADC0Value[1];
   x1 = (x1+1)% WINDOW_SIZE;
   average1 = sum1 / WINDOW_SIZE;
    digit[4] = average / 1000;
   digit[5] = average / 100 % 10;
   digit[6] = average / 10 % 10;
   digit[7] = average % 10;
```

```
            digit[0] = average1 / 1000;

            digit[1] = average1 / 100 % 10;

            digit[2] = average1 / 10 % 10;

            digit[3] = average1 % 10;

               clock10ms_flag = 0;

        }

            }

}

void GPIOInit(void)

{

            SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOK);
            while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOK)){};

            SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOM);
            while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOM)){};

            SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOC);
            while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOC)){};

            GPIOPinTypeGPIOOutput(GPIO_PORTK_BASE, GPIO_PIN_4|GPIO_PIN_5);

            GPIOPinTypeGPIOOutput(GPIO_PORTM_BASE, GPIO_PIN_0);

            GPIOPinTypeGPIOOutput(GPIO_PORTC_BASE, GPIO_PIN_7);

}

void ADCInit(void)

{


   SysCtlPeripheralEnable(SYSCTL_PERIPH_ADC0);


   SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);

   GPIOPinTypeADC(GPIO_PORTE_BASE, GPIO_PIN_3|GPIO_PIN_2);

   ADCSequenceConfigure(ADC0_BASE, 1, ADC_TRIGGER_PROCESSOR, 0);

   ADCSequenceStepConfigure(ADC0_BASE, 1, 0, ADC_CTL_CH0);
```

```
    ADCSequenceStepConfigure( ADC0_BASE, 1, 1,
                              ADC_CTL_CH1 | ADC_CTL_END | ADC_CTL_IE);
    ADCSequenceEnable(ADC0_BASE, 1);
    ADCIntClear(ADC0_BASE, 1);
}
void ADC_Sample(void)
{
    extern uint32_t pui32ADC0Value[4];
    ADCProcessorTrigger(ADC0_BASE, 1);
    while(!ADCIntStatus(ADC0_BASE, 1, false))
    {
    }
    ADCIntClear(ADC0_BASE, 1);
    ADCSequenceDataGet(ADC0_BASE, 1, pui32ADC0Value);
}
void SysTickInit(void)
{
    SysTickPeriodSet(ui32SysClock/SYSTICK_FREQUENCY);
    SysTickEnable();
    SysTickIntEnable();
}
void DevicesInit(void)
{
        ui32SysClock = SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ |SYSCTL_OSC_MAIN |
                                           SYSCTL_USE_PLL |SYSCTL_CFG_VCO_480),  20000000);
  GPIOInit();
```

```
  ADCInit();
  SysTickInit();
  IntMasterEnable();
}
void SysTick_Handler(void)
{
        if (++clock40ms >= V_T40ms)
        {
                clock40ms_flag = 1;
                clock40ms = 0;
        }
if (++clock10ms >= V_T10ms)
        {
                clock10ms_flag = 1;
                clock10ms = 0;   }
        if (++clock100ms >= V_T100ms)
        {
                clock100ms_flag = 1;
                clock100ms = 0;
        }
        TM1638_RefreshDIGIandLED(digit, pnt, led);
}
```

第二阶段

```
#include <stdint.h>
#include <stdbool.h>
```

```
#include "inc/hw_memmap.h"
#include "inc/hw_types.h"
#include "driverlib/debug.h"
#include "driverlib/gpio.h"
#include "driverlib/pin_map.h"
#include "driverlib/sysctl.h"
#include "driverlib/systick.h"
#include "driverlib/interrupt.h"
#include "driverlib/adc.h"
#include "tm1638.h"
#include "DAC6571.h"
#include <math.h>
#define SYSTICK_FREQUENCY                100
#define V_T10ms          1
#define V_T40ms          4
#define V_T100ms 10
#define V_T250ms 25
#define Iset_max 1200
#define WINDOW_SIZE 100
#define Vmax   5150
void GPIOInit(void);
void ADCInit(void);
```

```
void SysTickInit(void);

void DevicesInit(void);

void ADC_Sample(void);

void ADCHardwareOversampleConfigure(uint32_t ui32Base,uint32_t ui32Factor);


uint8_t clock40ms = 0;

uint8_t clock100ms = 0;

uint8_t clock10ms = 0;

uint8_t clock250ms = 0;

uint8_t clock40ms_flag = 0;

uint8_t  clock100ms_flag = 0;

uint8_t clock10ms_flag = 0;

uint8_t  clock250ms_flag = 0;

uint8_t clock250ms_enable = 0;

uint8_t digit[8]={' ',' ',' ',' ',' ',' ',' ',' '};


uint8_t pnt = 0x11;

uint8_t led[] = {0, 0, 0, 0, 0, 0, 0, 0};

uint32_t ui32SysClock;

uint32_t voltage_mV;


uint32_t current_mA;
```

```
uint32_t pui32ADC0Value[4];


uint32_t datas[WINDOW_SIZE] = {0};
                                                uint32_t datas1[WINDOW_SIZE] = {0};
                                                uint32_t sum = 0;
                                                uint32_t sum1 = 0;
                                                uint32_t average = 0;
                                                uint32_t average1 = 0;
                                                uint32_t x = 0;
                                                uint32_t x1 = 0;
uint8_t loadtype = 2;
uint8_t state = 1;
uint8_t key_code = 0;
uint8_t key_cnt = 0;
uint8_t state_change_flag=0;
uint32_t Iset = 0;
uint32_t DAC6571_code = 0;
uint8_t  DAC6571_flag = 1;
 int main(void)
{
        DevicesInit();
        while (clock100ms < 3);
```

```
        TM1638_Init();


        while (1)
        {
if (state == 1)
        {
          if (DAC6571_flag == 1)
          {
            DAC6571_flag = 0;
            pnt = 0x01;
            digit[4] = '_';
            digit[5] = key_code;
            digit[6] = '_';
            digit[7] = ' ';


            digit[0] = Iset / 1000 ;
            digit[1] = Iset / 100 % 10;
            digit[2] = Iset / 10 % 10;
            digit[3] = Iset % 10;


            DAC6571_code = floor(0.3187 * Iset - 27.029);
            DAC6571_Fastmode_Operation(DAC6571_code);
```

```
    }
    if (state_change_flag == 1)
    {
        state_change_flag = 0;
        clock250ms_enable = 1;
        state = 2;
    }
}

else if (state == 2)
{
    if (clock250ms_flag == 1)
    {
        clock250ms_flag = 0;
        x = 0;
    x1 = 0;
        loadtype = 2;
        state = 3;
    }
}
else if (state == 3)
{
```

```
            if (DAC6571_flag == 1)
            {

               state = 1;
            }
            else
            {

               if (clock10ms_flag == 1)
               {
                  clock10ms_flag = 0;
                              pnt = 0x11;
                ADC_Sample();
pui32ADC0Value[0] =(uint32_t)pui32ADC0Value[0]*3300*5/3/4095-2;
pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
          sum += pui32ADC0Value[0];
                  sum -= datas[x];
                  datas[x] = pui32ADC0Value[0];
                  x = (x+1)%WINDOW_SIZE;
                  average = sum / WINDOW_SIZE;
          sum1 += pui32ADC0Value[1];
                  sum1 -= datas1[x1];
```

```
            datas1[x1] = pui32ADC0Value[1];
            x1 = (x1+1)% WINDOW_SIZE;
         average1 = sum1 / WINDOW_SIZE;
        digit[4] = average / 1000;
         digit[5] = average / 100 % 10;
            digit[6] = average / 10 % 10;
            digit[7] = average % 10;

            digit[0] = average1 / 1000;
            digit[1] = average1 / 100 % 10;
            digit[2] = average1 / 10 % 10;
            digit[3] = average1 % 10;

            if (average < 4 * average1) loadtype = 1;
            else if (average > 8 * average1) loadtype = 3;
            else loadtype = 2;

if (loadtype == 1){

        pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
            sum1 += pui32ADC0Value[1];
            sum1 -= datas1[x1];
```

```
                datas1[x1] = pui32ADC0Value[1];
                x1 = (x1+1)% WINDOW_SIZE;
               average1 = (uint32_t)sum1 / WINDOW_SIZE*1.009+16;
             digit[0] = average1 / 1000;
              digit[1] = average1 / 100 % 10;
                digit[2] = average1 / 10 % 10;
                digit[3] = average1 % 10;
                 }
if (loadtype == 2){
        pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
                sum1 += pui32ADC0Value[1];
                sum1 -= datas1[x1];
                datas1[x1] = pui32ADC0Value[1];
                x1 = (x1+1)% WINDOW_SIZE;
                                        average1 = sum1 / WINDOW_SIZE*1.019+15;
                                  digit[0] = average1 / 1000;
             digit[1] = average1 / 100 % 10;
                digit[2] = average1 / 10 % 10;
                digit[3] = average1 % 10;
                }

if (loadtype == 3){
        pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
```

```
            sum1 += pui32ADC0Value[1];
            sum1 -= datas1[x1];
            datas1[x1] = pui32ADC0Value[1];
            x1 = (x1+1)% WINDOW_SIZE;
           average1 = sum1 / WINDOW_SIZE*1.048+13;
               digit[0] = average1 / 1000;
             digit[1] = average1 / 100 % 10;
            digit[2] = average1 / 10 % 10;
            digit[3] = average1 % 10;
        }
                                          }
              if (average >= Vmax)
                        {
                 loadtype = 2;
                 state = 4;
              }
    }
}
else if (state == 4)
{
    if (clock10ms_flag == 1)
    {
```

```
            clock10ms_flag = 0;
          ADC_Sample();
pui32ADC0Value[0] =(uint32_t)pui32ADC0Value[0]*3300/4095*5/3+10;
pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095+6;
           sum += pui32ADC0Value[0];
            sum -= datas[x];
            datas[x] = pui32ADC0Value[0];
            x = (x+1)%WINDOW_SIZE;
            average = sum / WINDOW_SIZE;
           sum1 += pui32ADC0Value[1];
            sum1 -= datas1[x1];
            datas1[x1] = pui32ADC0Value[1];
            x1 = (x1+1)% WINDOW_SIZE;
           average1 = sum1 / WINDOW_SIZE;
            digit[4] = average / 1000;
             digit[5] = average / 100 % 10;
            digit[6] = average / 10 % 10;
            digit[7] = average % 10;
            digit[0] = average1 / 1000;
            digit[1] = average1 / 100 % 10;
            digit[2] = average1 / 10 % 10;
            digit[3] = average1 % 10;
```

```
            if (average < 4 * average1) loadtype = 1;
                else if (average > 7 * average1) loadtype = 3;
                else loadtype = 2;
if (loadtype == 1){

        pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
                sum1 += pui32ADC0Value[1];
                sum1 -= datas1[x1];
                datas1[x1] = pui32ADC0Value[1];
                x1 = (x1+1)% WINDOW_SIZE;
                average1 = (uint32_t)sum1 / WINDOW_SIZE*1.009+17+70;
                  digit[0] = average1 / 1000;
                   digit[1] = average1 / 100 % 10;
                digit[2] = average1 / 10 % 10;
                digit[3] = average1 % 10;
                 }

 if (loadtype == 2){

        pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
                sum1 += pui32ADC0Value[1];
                sum1 -= datas1[x1];
                datas1[x1] = pui32ADC0Value[1];
```

```
                x1 = (x1+1)% WINDOW_SIZE;
                 average1 = sum1 / WINDOW_SIZE*1.019+15+71;
                   digit[0] = average1 / 1000;
                digit[1] = average1 / 100 % 10;
                digit[2] = average1 / 10 % 10;
                digit[3] = average1 % 10;
                }
                                                                    if
(loadtype == 3){
          pui32ADC0Value[1] = (uint32_t)pui32ADC0Value[1]*3300/4095;
                 sum1 += pui32ADC0Value[1];
                 sum1 -= datas1[x1];
                 datas1[x1] = pui32ADC0Value[1];
                 x1 = (x1+1)% WINDOW_SIZE;
                average1 = sum1 / WINDOW_SIZE*1.048+13+50;
                digit[0] = average1 / 1000;
                 digit[1] = average1 / 100 % 10;
                 digit[2] = average1 / 10 % 10;
                 digit[3] = average1 % 10;
              }
                                                                    }
```

```
            if (average1 < Iset && average > Vmax) state = 4;
            else
            {
               x = 0;
              x1=0;
               loadtype = 2;
               state = 3;
            }
          }
        }
          }
void GPIOInit(void)
{
        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOK);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOK)){};
        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOM);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOM)){};

        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOC);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOC)){};
```

```
        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOL);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOL)){};

        GPIOPinTypeGPIOOutput(GPIO_PORTK_BASE, GPIO_PIN_4|GPIO_PIN_5);

        GPIOPinTypeGPIOOutput(GPIO_PORTM_BASE, GPIO_PIN_0);

        GPIOPinTypeGPIOOutput(GPIO_PORTC_BASE, GPIO_PIN_7);
        GPIOPinTypeGPIOOutput(GPIO_PORTL_BASE, GPIO_PIN_0|GPIO_PIN_1);
}
void ADCInit(void)
{
   SysCtlPeripheralEnable(SYSCTL_PERIPH_ADC0);
   SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);

   GPIOPinTypeADC(GPIO_PORTE_BASE, GPIO_PIN_3|GPIO_PIN_2);
   ADCSequenceConfigure(ADC0_BASE, 1, ADC_TRIGGER_PROCESSOR, 0);
   ADCSequenceStepConfigure(ADC0_BASE, 1, 0, ADC_CTL_CH0);
   ADCSequenceStepConfigure( ADC0_BASE, 1, 1,
                        ADC_CTL_CH1 | ADC_CTL_END | ADC_CTL_IE);
   ADCSequenceEnable(ADC0_BASE, 1);
   ADCIntClear(ADC0_BASE, 1);
```

```
}
void ADC_Sample(void)
{
   extern uint32_t pui32ADC0Value[4];
   ADCProcessorTrigger(ADC0_BASE, 1);
   while(!ADCIntStatus(ADC0_BASE, 1, false))
   {
   }
   ADCIntClear(ADC0_BASE, 1);



   ADCSequenceDataGet(ADC0_BASE, 1, pui32ADC0Value);


}


void SysTickInit(void)
{
   SysTickPeriodSet(ui32SysClock/SYSTICK_FREQUENCY);
   SysTickEnable();
   SysTickIntEnable();
}
void DevicesInit(void)
```

```
{
        ui32SysClock = SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ |SYSCTL_OSC_MAIN |
                                SYSCTL_USE_PLL |SYSCTL_CFG_VCO_480),
                                20000000);

 GPIOInit();
 ADCInit();
 SysTickInit();
 IntMasterEnable();
}
void SysTick_Handler(void)
{


        if (++clock10ms >= V_T10ms)
        {
                clock10ms_flag = 1;
                clock10ms = 0;
        }

        if (++clock100ms >= V_T100ms)
        {
```

```
                clock100ms_flag = 1;
                clock100ms = 0;
        }

if (clock250ms_enable == 1)
{
   if (++clock250ms >= V_T250ms)
   {
      clock250ms_flag = 1;
      clock250ms_enable = 0;
      clock250ms = 0;
   }
}
        TM1638_RefreshDIGIandLED(digit, pnt, led);
        key_code = TM1638_Readkeyboard();
        if (key_code != 0)
        {
                if (key_cnt < 4) key_cnt++;
                else if (key_cnt == 4)
                {

      switch(key_code)
```

```
{
   case 1:
    if (Iset < Iset_max - 100)
            {
 Iset += 100;
 DAC6571_flag = 1;
            }
      break;
   case 4:
      if (Iset > 100)
 {
  Iset -= 100;
 DAC6571_flag = 1;
                                  }
      break;
   case 2:
     if (Iset < Iset_max - 10)
 {
 Iset += 10;
 DAC6571_flag = 1;
                                  }
      break;
```

```
  case 5:
     if (Iset >= 10)
{
 Iset -= 10;
DAC6571_flag = 1;
                                    }
      break;
  case 3:
     if (Iset < Iset_max - 1)
{
Iset += 1;
DAC6571_flag = 1;
                                    }
      break;
  case 6:
     if (Iset >= 1)
 {
 Iset -= 1;
 DAC6571_flag = 1;
                                    }
      break;
case 8:
```

```
            state_change_flag = 1;
           default:
             break;
        }
                                key_cnt = 5;                }
         }
        else key_cnt = 0;
}
```

第三阶段

```
#include <stdint.h>
#include <stdbool.h>
#include "inc/hw_memmap.h"
#include "inc/hw_types.h"
#include "driverlib/debug.h"
#include "driverlib/gpio.h"
#include "driverlib/pin_map.h"
#include "driverlib/sysctl.h"
#include "driverlib/systick.h"
#include "driverlib/interrupt.h"
#include "driverlib/adc.h"
#include "tm1638.h"
#include "DAC6571.h"
#include <math.h>
```

```
#define SYSTICK_FREQUENCY                100

#define V_T10ms          1

#define V_T40ms          4

#define V_T100ms 10


#define Iset_max 1200

#define WINDOW_SIZE 100

#define Vmax   5150

void GPIOInit(void);

void ADCInit(void);

void SysTickInit(void);

void DevicesInit(void);

uint32_t ADC_Sample(uint32_t Sam_num);

void ADCHardwareOversampleConfigure(uint32_t ui32Base,uint32_t ui32Factor);

uint8_t clock40ms = 0;

uint8_t clock100ms = 0;

uint8_t clock10ms = 0;

uint8_t clock250ms = 0;


uint8_t clock40ms_flag = 0;

uint8_t  clock100ms_flag = 0;

uint8_t clock10ms_flag = 0;

uint8_t clock100ms_enable = 0;

uint8_t digit[8]={' ',' ',' ',' ',' ',' ',' ',' '};

uint8_t pnt = 0x11;

uint8_t led[] = {0, 0, 0, 0, 0, 0, 0, 0};
```

```
uint32_t ui32SysClock;
uint32_t voltage_mV;
uint32_t current_mA;

uint32_t pui32ADC0Value[4];

uint32_t datas[WINDOW_SIZE] = {0};
                                        uint32_t datas1[WINDOW_SIZE] = {0};
                                        uint32_t datas2[WINDOW_SIZE] = {0};
                                        uint32_t sum = 0;
                                        uint32_t sum1 = 0;
                                        uint32_t sum2 = 0;
                                        uint32_t average = 0;
                                        uint32_t average1 = 0;
                                        uint32_t average2 = 0;
                                        uint32_t x = 0;
                                        uint32_t x1 = 0;
                                        uint32_t x2 = 0;
uint8_t loadtype = 2;
uint8_t state = 1;
uint8_t key_code = 0;
uint8_t key_cnt = 0;
uint8_t state_change_flag=0;
uint32_t Iset = 0;
uint32_t DAC6571_code = 0;
uint8_t  DAC6571_flag = 1;
uint32_t display = 0;
```

```
 int main(void)
{
        DevicesInit();

        while (clock100ms < 3);
        TM1638_Init();

        while (1)
        {

       if (state == 1)
       {



          if (clock10ms_flag == 1)
          {
             clock10ms_flag = 0;

                                                pui32ADC0Value[0] =
ADC_Sample(3)*3300/4095*5/3;
                                                pui32ADC0Value[1] =
ADC_Sample(1)*3300/4095*10/15;
                pui32ADC0Value[2] = ADC_Sample(2)*3300/4095;
                                          sum += pui32ADC0Value[0];
                sum -= datas[x];
                datas[x] = pui32ADC0Value[0];
                x = (x+1)%WINDOW_SIZE;
```

```
average = sum / WINDOW_SIZE;
                                        sum1 += pui32ADC0Value[1];
sum1 -= datas1[x1];
datas1[x1] = pui32ADC0Value[1];
x1 = (x1+1)% WINDOW_SIZE;
                                        average1 = sum1 / WINDOW_SIZE;
                                                                    sum2 +=
pui32ADC0Value[2];
sum2 -= datas2[x2];
datas2[x2] = pui32ADC0Value[2];
x2 = (x2+1)%WINDOW_SIZE;
average2 = sum2 / WINDOW_SIZE;
                                                                if(display==0)
                                                                    {
                                                                      pnt = 0x11;
                                        digit[4] = average1 / 1000;
digit[5] = average1 / 100 % 10;
digit[6] = average1 / 10 % 10;
digit[7] = average1 % 10;

digit[0] = average2 / 1000;
digit[1] = average2 / 100 % 10;
digit[2] = average2 / 10 % 10;
digit[3] = average2 % 10;

if (average < 4 * average2) loadtype = 1;
else if (average > 8 * average2) loadtype = 3;
else loadtype = 2;
```

```
                                                                    if
(loadtype == 1){

        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[1];
            x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = (uint32_t)sum2 / WINDOW_SIZE*1.014+7;
                                            digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
             }
                                                                    if (loadtype == 2){

        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[2];
            x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = sum2 / WINDOW_SIZE*1.019+15;
                                            digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
            }
                                                                    if
(loadtype == 3){
```

```
        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[2];
            x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = sum2 / WINDOW_SIZE*1.022+11;
                                                digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
                                                                                }
                                                                                }
                                                                    else
                                                                        {
                                                                           pnt = 0x10;
                                                                      digit[4] = average / 1000;
            digit[5] = average / 100 % 10;
            digit[6] = average / 10 % 10;
            digit[7] = average % 10;
                                                                          digit[0] = digit[1] =
digit[2] = digit[3] = ' ';
                                                                    }
          }

          if ( x == WINDOW_SIZE )
          {
             Iset = ( average1 + average2 ) / 2;
```

```
        DAC6571_code = floor(0.3076 * Iset - 26.487);
        DAC6571_Fastmode_Operation(DAC6571_code);

        clock100ms_enable = 1;
        state = 2;

    }
}

else if (state == 2)
{
    if (clock100ms_flag == 1)
    {
        clock100ms_flag = 0;
        x = 0;
                                                        x1=0;
                                                        x2=0;
        state = 3;
    }
}

else if (state == 3)
{

    if (clock10ms_flag == 1)
    {
```

```
            clock10ms_flag = 0;
              pui32ADC0Value[0] = ADC_Sample(3)*3300/4095*5/3;
                                                  pui32ADC0Value[1] =
ADC_Sample(1)*3300/4095*10/15;
              pui32ADC0Value[2] = ADC_Sample(2)*3300/4095;
                                        sum += pui32ADC0Value[0];
              sum -= datas[x];
              datas[x] = pui32ADC0Value[0];
              x = (x+1)%WINDOW_SIZE;
              average = sum / WINDOW_SIZE;
                                        sum1 += pui32ADC0Value[1];
              sum1 -= datas1[x1];
              datas1[x1] = pui32ADC0Value[1];
              x1 = (x1+1)% WINDOW_SIZE;
                                        average1 = sum1 / WINDOW_SIZE;
                                                            sum2 +=
pui32ADC0Value[2];
              sum2 -= datas2[x2];
              datas2[x2] = pui32ADC0Value[2];
              x2 = (x2+1)%WINDOW_SIZE;
              average2 = sum2 / WINDOW_SIZE;
                                                        if(display==0)
                                                              {
                                                                pnt = 0x11;
                                        digit[4] = average1 / 1000;
              digit[5] = average1 / 100 % 10;
              digit[6] = average1 / 10 % 10;
              digit[7] = average1 % 10;
```

```
            digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;

            if (average < 4 * average2) loadtype = 1;
            else if (average > 8 * average2) loadtype = 3;
            else loadtype = 2;
                                                                    if
(loadtype == 1){
        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[1];
            x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = (uint32_t)sum2 / WINDOW_SIZE*1.014+7;
                                            digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
             }
                                                                  if (loadtype == 2){
        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[2];
```

```
            x2 = (x2+1)% WINDOW_SIZE;
                                        average2 = sum2 / WINDOW_SIZE*1.019+15;
                                                    digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
            }
                                                                                    if
(loadtype == 3){

        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[2];
            x2 = (x2+1)% WINDOW_SIZE;
                                        average2 = sum2 / WINDOW_SIZE*1.022+11;
                                                    digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
                                                                                    }
                                                                                    }
                                                                        else
                                                                            {
                                                                                pnt = 0x10;
                                                                        digit[4] = average / 1000;

            digit[5] = average / 100 % 10;
            digit[6] = average / 10 % 10;
```

```
                digit[7] = average % 10;
                                                                digit[0] = digit[1] =
digit[2] = digit[3] = ' ';
                                                            }
            }

            if ( x == WINDOW_SIZE )
            {
               if (average2 > average1 + 100 || average1 > average2 + 100)
               {
                  state = 1;
               }
               else if (average1 < 100 && average < Vmax)
               {
                  state = 4;
               }
               else
               {
                  if (average2 > average1 + 4)
                  {
                     DAC6571_code--;
                     DAC6571_Fastmode_Operation(DAC6571_code);
                     clock100ms_enable = 1;
                     state = 2;
                  }
                  else if (average2 < average1 - 4)
                  {
                     DAC6571_code++;
```

```
                DAC6571_Fastmode_Operation(DAC6571_code);
                clock100ms_enable = 1;
                state = 2;
              }
            }
          }
        }

        else if (state == 4)
        {
          if (clock10ms_flag == 1)
          {
            clock10ms_flag = 0;
             pui32ADC0Value[0] = ADC_Sample(3)*3300/4095*5/3;
                                                    pui32ADC0Value[1] =
ADC_Sample(1)*3300/4095*10/15;
                pui32ADC0Value[2] = ADC_Sample(2)*3300/4095;
                                          sum += pui32ADC0Value[0];
                sum -= datas[x];
                datas[x] = pui32ADC0Value[0];
                x = (x+1)%WINDOW_SIZE;
                average = sum / WINDOW_SIZE;
                                          sum1 += pui32ADC0Value[1];
                sum1 -= datas1[x1];
                datas1[x1] = pui32ADC0Value[1];
                x1 = (x1+1)% WINDOW_SIZE;
                                          average1 = sum1 / WINDOW_SIZE;
```

```
                                                    sum2 +=
pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[2];
            x2 = (x2+1)%WINDOW_SIZE;
            average2 = sum2 / WINDOW_SIZE;
                                                  if(display==0)
                                                      {
                                                        pnt = 0x11;
                                        digit[4] = average1 / 1000;
            digit[5] = average1 / 100 % 10;
            digit[6] = average1 / 10 % 10;
            digit[7] = average1 % 10;

            digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;

            if (average < 4 * average2) loadtype = 1;
            else if (average > 8 * average2) loadtype = 3;
            else loadtype = 2;
                                                                  if
(loadtype == 1){
        pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
            sum2 += pui32ADC0Value[2];
            sum2 -= datas2[x2];
            datas2[x2] = pui32ADC0Value[1];
```

```
x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = (uint32_t)sum2 / WINDOW_SIZE*1.014+7;
                                                digit[0] = average2 / 1000;
digit[1] = average2 / 100 % 10;
digit[2] = average2 / 10 % 10;
digit[3] = average2 % 10;
 }
                                                                    if (loadtype == 2){
pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
sum2 += pui32ADC0Value[2];
sum2 -= datas2[x2];
datas2[x2] = pui32ADC0Value[2];
x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = sum2 / WINDOW_SIZE*1.019+15;
                                                digit[0] = average2 / 1000;
digit[1] = average2 / 100 % 10;
digit[2] = average2 / 10 % 10;
digit[3] = average2 % 10;
}
                                                                                if
(loadtype == 3){
pui32ADC0Value[2] = (uint32_t)pui32ADC0Value[2]*3300/4095;
sum2 += pui32ADC0Value[2];
sum2 -= datas2[x2];
datas2[x2] = pui32ADC0Value[2];
x2 = (x2+1)% WINDOW_SIZE;
                                    average2 = sum2 / WINDOW_SIZE*1.022+11;
```

```
                                                    digit[0] = average2 / 1000;
            digit[1] = average2 / 100 % 10;
            digit[2] = average2 / 10 % 10;
            digit[3] = average2 % 10;
                                                                                }
                                                                                }
                                                          else //ÏÔÊ¾µçÑ¹
                                                                {
                                                                   pnt = 0x10;
                                                              digit[4] = average / 1000;
            digit[5] = average / 100 % 10;
            digit[6] = average / 10 % 10;
            digit[7] = average % 10;
                                                                    digit[0] = digit[1] =
digit[2] = digit[3] = ' ';
                                                          }
          }

          if (average1 < 100 && average < Vmax) state = 4;
          else
          {
             x=0;
                                                            x1=0;
                                                            x2=0;
             state = 3;
          }
        }
```

```
    }
  }
void GPIOInit(void)
{

        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOK);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOK)){};

        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOM);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOM)){};
        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOC);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOC)){};



        SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOL);
        while(!SysCtlPeripheralReady(SYSCTL_PERIPH_GPIOL)){};

        GPIOPinTypeGPIOOutput(GPIO_PORTK_BASE, GPIO_PIN_4|GPIO_PIN_5);
        GPIOPinTypeGPIOOutput(GPIO_PORTM_BASE, GPIO_PIN_0);
        GPIOPinTypeGPIOOutput(GPIO_PORTC_BASE, GPIO_PIN_7);
        GPIOPinTypeGPIOOutput(GPIO_PORTL_BASE, GPIO_PIN_0|GPIO_PIN_1);
}

void ADCInit(void)
{
  SysCtlPeripheralEnable(SYSCTL_PERIPH_ADC0);
  SysCtlPeripheralEnable(SYSCTL_PERIPH_GPIOE);
```

```
    GPIOPinTypeADC(GPIO_PORTE_BASE, GPIO_PIN_1);
    GPIOPinTypeADC(GPIO_PORTE_BASE, GPIO_PIN_0);
    GPIOPinTypeADC(GPIO_PORTE_BASE, GPIO_PIN_2);
    ADCSequenceConfigure(ADC0_BASE, 3, ADC_TRIGGER_PROCESSOR, 0);
    ADCSequenceConfigure(ADC0_BASE, 2, ADC_TRIGGER_PROCESSOR, 1);
    ADCSequenceConfigure(ADC0_BASE, 1, ADC_TRIGGER_PROCESSOR, 2);
    ADCSequenceStepConfigure(ADC0_BASE, 3, 0, ADC_CTL_CH2 | ADC_CTL_IE |
                             ADC_CTL_END);
    ADCSequenceStepConfigure(ADC0_BASE, 2, 0, ADC_CTL_CH3 | ADC_CTL_IE |
                             ADC_CTL_END);
    ADCSequenceStepConfigure(ADC0_BASE, 1, 0, ADC_CTL_CH1 | ADC_CTL_IE |
                    ADC_CTL_END);

    ADCSequenceEnable(ADC0_BASE, 3);
    ADCSequenceEnable(ADC0_BASE, 2);
    ADCSequenceEnable(ADC0_BASE, 1);

    ADCIntClear(ADC0_BASE, 3);
    ADCIntClear(ADC0_BASE, 2);
    ADCIntClear(ADC0_BASE,1);
}

uint32_t ADC_Sample(uint32_t Sam_num)
{
    uint32_t pui32ADC0Value[1];
    ADCProcessorTrigger(ADC0_BASE, Sam_num);
    while(!ADCIntStatus(ADC0_BASE, Sam_num, false)) { }
```

```
    ADCIntClear(ADC0_BASE, Sam_num);
    ADCSequenceDataGet(ADC0_BASE, Sam_num, pui32ADC0Value);
    return pui32ADC0Value[0];
}

void SysTickInit(void)
{
    SysTickPeriodSet(ui32SysClock/SYSTICK_FREQUENCY);
    SysTickEnable();
    SysTickIntEnable();
}

void DevicesInit(void)
{

        ui32SysClock = SysCtlClockFreqSet((SYSCTL_XTAL_25MHZ |SYSCTL_OSC_MAIN |
                                           SYSCTL_USE_PLL |SYSCTL_CFG_VCO_480),
                                           20000000);

  GPIOInit();
  ADCInit();
  SysTickInit();
  IntMasterEnable();
}

void SysTick_Handler(void)
{
```

```
        if (++clock10ms >= V_T10ms)
        {
                clock10ms_flag = 1;
                clock10ms = 0;
        }



        if (++clock100ms >= V_T100ms)
        {
                clock100ms_flag = 1;
                clock100ms = 0;
        }


if (clock100ms_enable == 1)
{
   if (++clock100ms >= V_T100ms)
   {
      clock100ms_flag = 1;
      clock100ms_enable = 0;
      clock100ms = 0;
   }
}
```

```
 TM1638_RefreshDIGIandLED(digit, pnt, led);


 key_code = TM1638_Readkeyboard();
 if (key_code != 0)
 {
            if (key_cnt < 4) key_cnt++;
            else if (key_cnt == 4)
            {


switch(key_code)
{
   case 1:
      if (Iset < Iset_max - 100)
                                    {
                                                Iset += 100;
                                                DAC6571_flag = 1;
                                    }
      break;
   case 4:
      if (Iset > 100)
                                    {
                                                Iset -= 100;
                                                DAC6571_flag = 1;
                                    }
      break;
   case 2:
```

```
            if (Iset < Iset_max - 10)
                                    {
                                        Iset += 10;
                                        DAC6571_flag = 1;
                                    }
             break;
           case 5:
             if (Iset >= 10)
                                    {
                                        Iset -= 10;
                                        DAC6571_flag = 1;
                                    }
             break;
           case 3:
             if (Iset < Iset_max - 1)
                                    {
                                        Iset += 1;
                                        DAC6571_flag = 1;
                                    }
             break;
           case 6:
             if (Iset >= 1)
                                    {
Iset -= 1;
  DAC6571_flag = 1;
                                    }
             break;
```

```
            case 8:
              state_change_flag = 1;
            display = (display+1)%2;
              default:
              break;
          }
          key_cnt = 5;
                    }
}
else key_cnt = 0;
}
```

13.